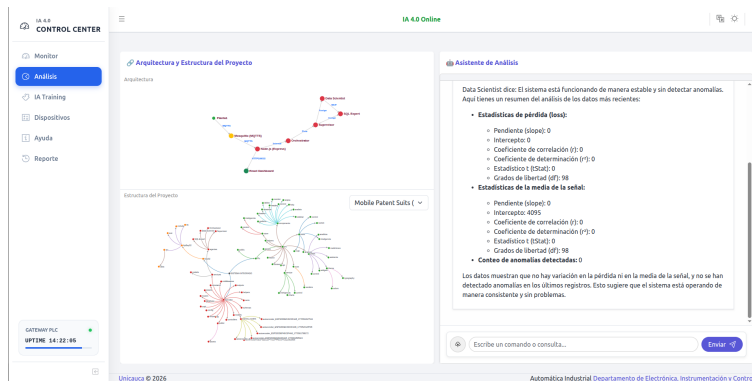


IA en la Industria 4.0

Fabio Hernán Realpe Martínez



UNIVERSIDAD DEL CAUCA
INGENIERIA EN AUTOMATICA INDUSTRIAL
POPAYAN - 2026

Índice general

1. Herramientas de IA en la Industria 4.0	9
1.1. Unidad temática: Arquitecturas de Orquestación Agéntica para la Industria 4.0	9
1.1.1. La Capa de Orquestación: El Sistema Nervioso Central	9
1.1.2. MCP y el Vínculo con el Hardware	10
1.1.3. Interfaz de Usuario como Reflejo del Razonamiento	11
1.2. Que es un agente?	14
1.2.1. Memoria de Corto Plazo (Contexto Operativo):	14
1.2.2. Memoria de Largo Plazo (Historial y Patrones):	14
1.2.3. Razonamiento mediante Reglas de Negocio:	15
1.2.4. Diferenciación Técnica: El Agente como Operador Dinámico	15
1.3. LangChain	16
1.4. LangGraph	16
1.4.1. LangGraph como el "Cerebro" de la Industria 4.0	17
1.4.2. Ingeniería de Prompts: Del Texto a la Estructura de Datos	18
1.5. Tokens	18
1.6. Modelos de Chat vs. Herramientas (Tool/JSON)	22
1.7. Zod?	23
1.8. Estructura de salida	25
2. Servidor	27
2.1. Capa de Inicio — app.js	29
2.2. Núcleo del Servidor — server.js	30
2.3. Capa de Comunicación — Express (API REST)	31
2.4. Capa de Enrutamiento — Routers	31
2.5. Capa de Control — Controllers	32
2.6. Capa de Lógica de Negocio — Servicios	32
2.7. Capa de Herramientas — MCP Service y Helpers	33
2.8. Capa de Persistencia — Base de Datos y Modelos	34
2.9. Capa de Comunicación en Tiempo Real — MQTT y WebSockets	34

3. Agentes	35
3.1. Modelos de IA locales	35
3.2. Servicio Multi agente	37
4. Certificado TLS, MQTTS	40
4.1. Guía de instalación y configuración de Mosquitto MQTT Broker	40
4.2. Procedimiento de configuración de TLS y WebSocket seguro . .	43
4.2.1. Generación de la Autoridad Certificadora (CA)	43
4.3. Configuración de Mosquitto con WebSocket y Seguridad	44
4.3.1. Introducción	44
4.3.2. Archivo de Configuración Principal	45
4.3.3. Descripción de Directivas de Configuración	46
4.4. Web Socket	48
5. Esp32	50
5.1. Qué es PlatformIO? PlatformIO	50
5.2. Esquema del Proyecto	55
5.3. Programación de protocolos de comunicación	58
5.3.1. Configuración de variables globales para las conexión wiffi	58
5.3.2. Configuración de Lectura, Escritura y Reset de parámetros.	60
5.3.3. Ciclo de Vida del MQtt	64

Índice de figuras

1.1. Arquitectura de Tres Capas para el Control Autónomo de Procesos	12
2.1. Servidor	28
3.1. Diagrama Multi agente	38
3.2. Lang Graph Orquestador	39
5.1. Proyecto MCP	51
5.2. Estructura del proyecto	51
5.3. Diagrama principal del proyecto	56
5.4. Ciclo de Vida de un Mensaje MQTT.	58
5.5. Arquitectura de configuración Wiffi	61
5.6. Ciclo de vida de Mqtt	65

Algoritmos

1.1. Esquema de validación Zod.	26
1.2. Respuesta del modelo.	26
2.1. variables de entorno	29
2.2. Arranque del sistema	30
2.3. Sistema Servidor	31
2.4. Comunicación HTTP	31
2.5. Enrutamiento	32
2.6. Controladores	32
2.7. Lógica de Negocio	33
2.8. Herramientas MCP	33
2.9. Persistencia de Datos	34
2.10. Comunicación en Tiempo Real	34
3.1. Configuración de modelos locales	36
4.1. Configuración <code>mosquitto.conf</code>	46
5.1. <code>platformio.ini</code>	52
5.2. Compilar y cargar	52
5.3. Configuración de <code>platformio.in</code>	53
5.4. Versionamiento	55
5.5. Configuraciones globales del dispositivo	59
5.6. <code>mcp_settings.hpp</code>	62
5.7. <code>settingsRead</code>	62
5.8. <code>settingsReset</code>	63
5.9. <code>settingsSave</code>	64
5.10. Función <code>TaskMqttReconnect</code>	67
5.11. Función <code>mqttloop()</code>	68
5.12. Función <code>mqtt_connect()</code>	69
5.13. Función <code>callback()</code>	71
5.14. Función <code>mqtt_response()</code>	72
5.15. Función <code>mqtt_publish()</code>	73
5.16. <code>Json()</code>	74

Introduccion

La evolución industrial ha dado lugar a nuevos paradigmas tecnológicos que transforman la forma en que las organizaciones producen, gestionan y toman decisiones. En este contexto, surge el concepto de Industria 4.0, el cual representa una nueva etapa en el desarrollo de los sistemas productivos, caracterizada por la integración de tecnologías digitales, la interconexión de dispositivos y la automatización inteligente de los procesos. A continuación, se describen sus principales características organizadas por secciones fundamentales.

1. Digitalización e Interconectividad. Uno de los pilares más importantes de la Industria 4.0 es la digitalización de los procesos mediante el uso de sensores, dispositivos IoT y sistemas ciberfísicos capaces de capturar y transmitir información en tiempo real. Esta conectividad permite que máquinas, sistemas y plataformas digitales se comuniquen entre sí, generando entornos altamente automatizados e inteligentes. Gracias a esta interconexión, es posible supervisar procesos industriales de manera remota, mantener un flujo continuo de información y lograr una integración completa entre el mundo físico y el digital.
2. Automatización e Inteligencia Artificial. La Industria 4.0 se basa en la automatización avanzada mediante el uso de inteligencia artificial, algoritmos inteligentes y sistemas de control automatizados. Estas tecnologías permiten que los procesos industriales operen con mínima intervención humana, optimizando la eficiencia, reduciendo errores y mejorando la productividad. Además, los sistemas inteligentes pueden analizar información en tiempo real, identificar patrones y tomar decisiones autónomas para optimizar el funcionamiento de los procesos productivos.
3. Big Data y Análisis en Tiempo Real. Otro componente clave de la Industria 4.0 es el uso de Big Data, que permite recolectar, almacenar y procesar grandes volúmenes de información generada por dispositivos industriales. Este análisis avanzado facilita el monitoreo continuo, la identificación de tendencias y la predicción de fallas, lo que contribuye a mejorar la planificación, el mantenimiento predictivo y la toma de decisiones basada en datos. La capacidad de analizar información en tiempo real se convierte en un elemento esencial para lograr procesos más eficientes y confiables.

4. Seguridad y Comunicaciones Industriales. La seguridad es un aspecto fundamental dentro de la Industria 4.0, especialmente debido a la interconexión de múltiples dispositivos y sistemas. Para garantizar la protección de la información, se emplean protocolos de comunicación seguros, mecanismos de cifrado y sistemas de autenticación que aseguran la integridad, confidencialidad y disponibilidad de los datos. Estas tecnologías permiten mantener comunicaciones confiables entre los diferentes componentes del sistema industrial, reduciendo riesgos asociados a accesos no autorizados o fallas operativas.
5. Sistemas Ciberfísicos y Toma de Decisiones Inteligente. La Industria 4.0 se fundamenta en los sistemas ciberfísicos, los cuales integran elementos físicos con sistemas digitales para monitorear, analizar y controlar procesos industriales. Estos sistemas permiten la automatización descentralizada, donde las decisiones pueden tomarse de manera autónoma a partir del análisis de datos en tiempo real. Como resultado, se logra una mayor flexibilidad, adaptabilidad y eficiencia en los entornos productivos.

En conjunto, la Industria 4.0 representa una transformación profunda en los modelos industriales tradicionales, al integrar tecnologías digitales, automatización avanzada y análisis inteligente de datos. Este enfoque permite la creación de entornos productivos altamente conectados, eficientes y capaces de responder de manera dinámica a las necesidades del mercado y a los cambios en los procesos industriales.

El proyecto desarrollado evidencia una clara alineación con los principios de la Industria 4.0, ya que integra tecnologías de digitalización, conectividad, análisis inteligente y automatización dentro de una arquitectura tecnológica unificada. En primer lugar, cumple con el pilar de digitalización e interconectividad, al incorporar dispositivos IoT basados en microcontroladores que permiten la captura de datos desde sensores y actuadores en entornos físicos. Estos datos son transmitidos mediante protocolos industriales como MQTT y OPC UA, lo que garantiza la comunicación continua entre el mundo físico y el entorno digital, permitiendo la supervisión remota y la integración de múltiples sistemas dentro de una misma infraestructura.

Asimismo, el proyecto responde al pilar de automatización e inteligencia artificial, al implementar un backend inteligente basado en agentes capaces de analizar información, ejecutar consultas automatizadas y generar respuestas mediante modelos de inteligencia artificial. Este enfoque introduce capacidades de toma de decisiones asistidas, donde el sistema no solo recopila información, sino que también la interpreta y produce conocimiento útil para optimizar los procesos operativos. De esta manera, se evidencia una transición desde sistemas tradicionales de monitoreo hacia sistemas inteligentes capaces de adaptarse dinámicamente a los datos recibidos.

En relación con el pilar de Big Data y análisis en tiempo real, la solución contempla el almacenamiento estructurado de información en bases de datos modernas que permiten gestionar grandes volúmenes de datos históricos y en

tiempo real. Esto posibilita la aplicación de técnicas de análisis avanzado, identificación de patrones y generación de modelos predictivos que contribuyen a mejorar la eficiencia operativa, la planificación y el mantenimiento de los sistemas. El uso de herramientas de análisis de datos refuerza la capacidad del proyecto para transformar información en conocimiento estratégico.

Por otra parte, el sistema también cumple con el pilar de seguridad y comunicaciones industriales, mediante la implementación de protocolos seguros de transmisión como MQTTS y WebSockets cifrados, además del uso de certificados digitales y configuraciones de seguridad en los servicios. Estas medidas garantizan la protección de la información, la autenticidad de las comunicaciones y la integridad de los datos, aspectos esenciales en entornos industriales altamente interconectados.

Finalmente, el proyecto se alinea con el concepto de sistemas ciberfísicos, ya que integra dispositivos físicos, plataformas digitales, procesamiento inteligente y visualización en tiempo real dentro de una arquitectura unificada. Esta integración permite monitorear, analizar y controlar procesos de forma continua, logrando un entorno productivo inteligente, flexible y adaptativo. En conjunto, el proyecto no solo incorpora tecnologías propias de la Industria 4.0, sino que demuestra su aplicación práctica mediante una solución completa que conecta el mundo físico con la inteligencia digital, evidenciando una implementación integral de este paradigma industrial.

Capítulo 1

Herramientas de IA en la Industria 4.0

1.1. Unidad temática: Arquitecturas de Orquestación Agéntica para la Industria 4.0

Esta propuesta académica redefine la integración de la Inteligencia Artificial en entornos productivos, estableciendo una distinción crítica entre el consumo pasivo de modelos y el diseño de sistemas autónomos. A continuación, se explican los conceptos fundamentales de esta arquitectura bajo una visión de ingeniería agéntica:

La Superación del Modelo "Wrapper" El enfoque tradicional, conocido como "ChatGPT wrapper", se limita a actuar como una interfaz superficial que reenvía consultas a un modelo lingüístico sin poseer una lógica interna o conexión real con el entorno. En contraste, un sistema agéntico industrial se concibe como un operador proactivo. Mientras que el primero es un espejo estático de las capacidades del modelo, el sistema agéntico utiliza la IA para razonar, integrando el flujo de trabajo con la realidad física de la planta.

1.1.1. La Capa de Orquestación: El Sistema Nervioso Central

El núcleo de esta arquitectura es la Orquestación, implementada mediante LangGraph. Esta herramienta funciona como el cerebro del sistema, gestionando el estado y permitiendo flujos de decisión cíclicos. A diferencia de un chat convencional, esta capa dota a la IA de memoria y razonamiento lógico sobre procesos industriales, permitiendo que el sistema no solo responda, sino que mantenga la continuidad y coherencia en tareas de larga duración o procesos repetitivos de control.

1.1.2. MCP y el Vínculo con el Hardware

Para que la inteligencia lógica (LLM) pueda interactuar con la acción física (Hardware), se utiliza el Model Context Protocol (MCP). Este protocolo actúa como la capa de herramientas, permitiendo que los agentes de IA se conecten de manera segura y estandarizada con sensores IoT como sensores y bases de datos industriales. El MCP permite que la IA "sienta" el hardware, convirtiendo señales físicas en datos procesables para el razonamiento del agente.

MCP (Model Context Protocol) es un estándar abierto desarrollado recientemente por Anthropic que está cambiando la forma en que los asistentes de IA interactúan con los datos y las herramientas ver1.1.

- 1. Introducción En el panorama actual del desarrollo con Inteligencia Artificial, la integración de modelos de lenguaje (LLMs) con fuentes de datos externas (bases de datos, archivos locales, APIs) es fragmentada. Cada integración requiere una implementación personalizada, lo que genera silos de información y deuda técnica.

Model Context Protocol (MCP) surge como un estándar abierto diseñado para reemplazar estas integraciones punto a punto con un protocolo universal y seguro.

- 2. El Problema: El "Impuesto de Integración" Hasta ahora, conectar un Agente de IA a un entorno de trabajo implicaba:

Mantenimiento constante: APIs que cambian y requieren actualizaciones manuales.

Riesgos de seguridad: Métodos de acceso a datos no estandarizados.

Falta de portabilidad: Una herramienta diseñada para un modelo no funcionaba fácilmente en otro.

- 3. La Solución: Arquitectura MCP MCP funciona bajo un modelo de arquitectura Cliente-Servidor simplificado:

Componente	Función
MCP Host	La aplicación de IA (ej. Claude Desktop, IDEs, aplicaciones personalizadas en Node.js).
MCP Client	Mantiene la conexión bidireccional con los servidores dentro del Host.
MCP Server	Microservicios ligeros que exponen datos o herramientas específicas (Filesystem, GitHub, SQL, sensores) mediante el estándar MCP.

Tabla 1.1: Cliente-Servidor MCP.

- 4. Beneficios Clave Interoperabilidad:
 - Una vez que escribes un servidor MCP (por ejemplo, para leer datos de sensores o estados de un PLC), cualquier aplicación compatible con MCP puede usarlo.

- Control de Contexto: El modelo solo accede a la información necesaria en el momento preciso, optimizando el uso de la ventana de contexto.
- Seguridad Local: Los servidores pueden ejecutarse localmente, permitiendo que la IA interactúe con datos sensibles sin que estos salgan de la infraestructura privada del usuario o empresa.

5. Casos de Uso en Desarrollo

- Ingeniería de Software: Acceso directo a repositorios para refactorización de código en tiempo real.
- Sistemas Industriales / IoT: Un servidor MCP que actúe como puente entre protocolos industriales (Modbus/OPC-UA) y el modelo de IA para diagnóstico de fallas.
- Gestión de Datos: Consultas a bases de datos en lenguaje natural mediante la exposición de esquemas SQL seguros a través de MCP.

6. Conclusión y Próximos Pasos

- MCP no es solo una herramienta, es el puerto estándar que permitirá que los agentes de IA dejen de ser simples generadores de texto para convertirse en asistentes operativos reales con acceso a herramientas del mundo físico y digital.

1.1.3. Interfaz de Usuario como Reflejo del Razonamiento

Finalmente, la capa de UI, desarrollada en React o React Native, deja de ser una simple ventana de chat para convertirse en un tablero de control del proceso de pensamiento del agente. En esta arquitectura, la interfaz es un reflejo fiel de un proceso de razonamiento autónomo, donde el usuario puede observar cómo el agente decide y actúa sobre los activos físicos en tiempo real.

La figura muestra el esquema funcional del sistema ver 1.1.

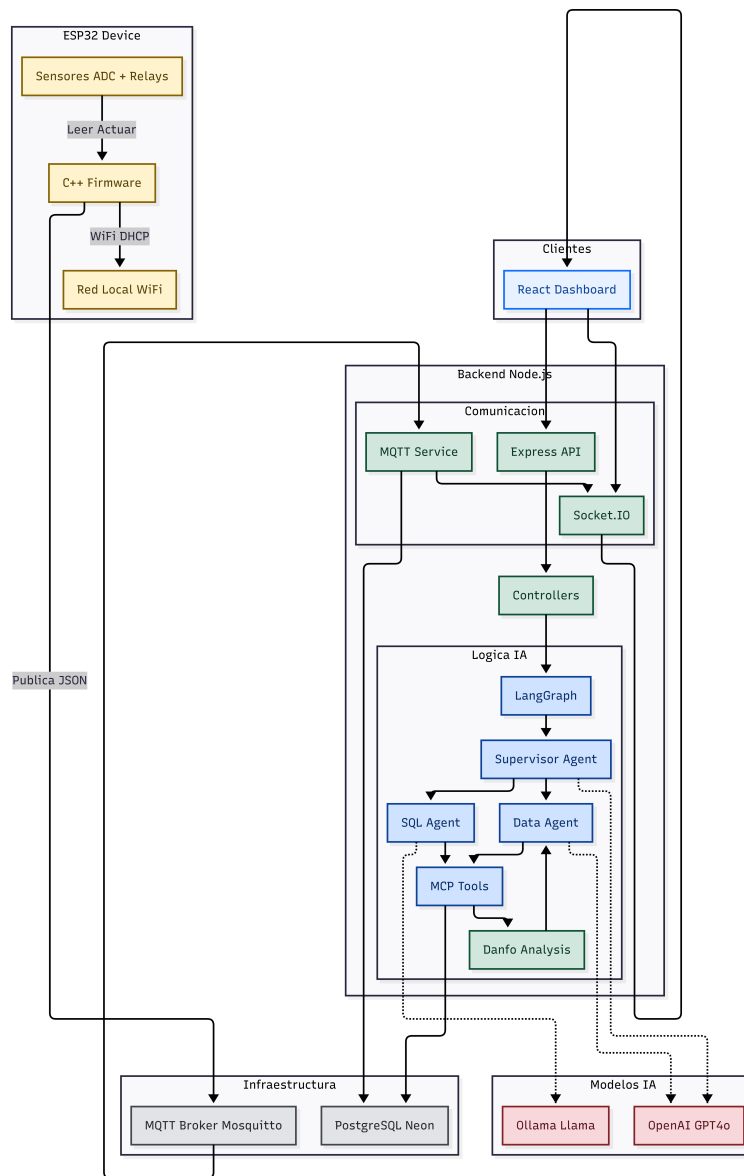


Figura 1.1: Arquitectura de Tres Capas para el Control Autónomo de Procesos

1. Capa de Interfaz (UI): El Espejo Reactivo En esta capa superior, el usuario interactúa con el sistema a través de una aplicación desarrollada en React Native. A diferencia de una aplicación convencional, esta interfaz no es solo un chat, sino un panel de control que refleja el razonamiento de la IA en tiempo real.

- WebSockets / Socket.io: Actúan como el puente de baja latencia que permite que el agente de IA envíe alertas o actualizaciones de sensores de forma instantánea a la pantalla del operador sin necesidad de recargar.
2. Capa de Orquestación: El Cerebro (LangGraph) Esta es la capa que trasciende el concepto de "wrapper". Aquí, LangGraph funciona como el sistema nervioso central que define la lógica de control.
 - Estado y Memoria: El sistema mantiene un registro de lo que ha sucedido en la planta, permitiendo que la IA tome decisiones basadas en el historial y no solo en el último mensaje recibido.
 - Flujos Cíclicos: Permite que el agente realice bucles de razonamiento: "Leo el sensor – > Analizo – > Si el valor es incorrecto, intento ajustar – > Vuelvo a leer".
 3. Capa de Models & Tools: El Vínculo con la Realidad Aquí es donde la inteligencia abstracta se encuentra con la ejecución física mediante el Model Context Protocol (MCP).

Servidor MCP: Es el estándar que permite al agente de IA "ver" y "tocar" el hardware. Expone las capacidades de la planta como herramientas que el modelo puede usar.

Protocolos Industriales:

- MQTT: Utilizado para la telemetría masiva de sensores IoT (como el sensor), permitiendo que el agente reciba datos de forma eficiente.
 - OPC-UA: Garantiza la interoperabilidad con maquinaria industrial pesada y servidores de planta estándar.
 - Modbus: Permite el control directo sobre PLCs y dispositivos que manejan la acción física (actuadores, motores).
4. Inteligencia y Datos LLM
 - (Llama3 / Ollama): Es el motor de inferencia que procesa el lenguaje natural y los datos de los sensores para generar planes de acción.
 - Bases de Datos (InfluxDB / Postgres): Almacenan el histórico de telemetría y los metadatos de la operación para que el agente pueda realizar análisis de tendencias o mantenimiento predictivo.

Conclusión del Flujo: Cuando un sensor reporta una anomalía vía MQTT, el Servidor MCP lo comunica al Grafo de Orquestación. El LLM analiza la situación, decide una acción correctiva (ej. ajustar una válvula vía Modbus) y actualiza simultáneamente la UI del operador mediante Sockets, transformando la planta mediante un ciclo cerrado de percepción y acción.

Para que el sistema sea eficiente, es vital distinguir cómo se procesa la información dentro de la orquestación:

1. Chain (Cadena): Sigue pasos fijos y predeterminados sin tomar decisiones propias. Se recomienda usarlas primero para tareas simples y lineales.
2. Agent (Agente): Tiene la capacidad de elegir el siguiente paso basándose en el contexto. Decide "qué hacer a continuación" para resolver problemas complejos, siempre bajo el control del desarrollador.

1.2. Que es un agente?

En esencia, un Agente de IA se define como la combinación de un LLM + Memoria + Herramientas + Reglas. Mientras que un wrapper es un espejo estático que solo refleja lo que la IA dice, un sistema agéntico industrial es un operador dinámico que utiliza estos componentes para transformar la planta:

Memoria e Integración: Gracias a la orquestación, el agente puede "pensar antes de actuar". Por ejemplo, si una regla de negocio dicta que no se debe aumentar la velocidad de una banda transportadora si la temperatura del motor supera un umbral crítico, el agente verifica el sensor mediante sus herramientas, razona la restricción usando su memoria y decide cancelar la acción, informando al operador el porqué de su decisión a través de la UI.

1.2.1. Memoria de Corto Plazo (Contexto Operativo):

El agente retiene los datos inmediatos del flujo de trabajo, como el estado actual de una línea de producción o la última lectura de un protocolo OPC-UA. Esto le permite "pensar" si la acción solicitada es coherente con el estado presente de la máquina

1.2.2. Memoria de Largo Plazo (Historial y Patrones):

Mediante la conexión con bases de datos como InfluxDB o PostgreSQL, el agente accede al histórico de fallos o métricas de rendimiento pasadas. Por ejemplo, si el agente detecta una vibración inusual, consulta su memoria para verificar si ese patrón precedió a una avería el mes anterior, activando un diagnóstico preventivo en lugar de una simple alerta.

En un sistema robusto, la memoria de largo plazo no vive dentro del modelo (LLM), sino que es una herramienta externa que el agente consulta a través de la capa de orquestación.

Su funcionamiento se divide en tres ejes:

1. Persistencia de Experiencias (Historial de Operación): A través de bases de datos como PostgreSQL o InfluxDB, el agente guarda cada evento crítico que ocurre en la planta. Si un motor presenta una vibración inusual, el agente consulta su memoria de largo plazo para verificar: "¿Este patrón de vibración ha ocurrido antes? ¿Terminó en una falla crítica la última vez?". Esto permite que el agente razone basándose en la experiencia acumulada.

2. RAG (Generación Aumentada por Recuperación) - El "Manual Técnico": La memoria de largo plazo también contiene el conocimiento estático: manuales de maquinaria, protocolos de seguridad y normativas industriales. Mediante técnicas de RAG, el agente busca en milisegundos la solución técnica a un problema detectado por los sensores, integrando ese conocimiento en su proceso de decisión.
3. Memoria Semántica vs. Memoria de Episodios:
Episódica: Recuerda interacciones pasadas con el operador o estados específicos de la planta.
Semántica: Comprende los conceptos técnicos y las relaciones entre los componentes de la industria 4.0.

1.2.3. Razonamiento mediante Reglas de Negocio:

La orquestación (vía LangGraph) permite al agente aplicar lógica de control compleja antes de actuar. Si una regla dicta que no se debe aumentar la velocidad de una banda transportadora si la temperatura del motor supera un umbral crítico, el agente verifica el sensor, razona la restricción y decide cancelar la operación, informando al operador el motivo técnico a través de la UI en React Native.

1.2.4. Diferenciación Técnica: El Agente como Operador Dinámico

Mientras que un wrapper es un espejo estático que solo refleja lo que la IA dice sin comprender el entorno, un sistema agéntico industrial se define como la combinación de LLM + Memoria + Herramientas + Reglas. Este sistema utiliza:

1. La IA para Razonar: Procesa datos complejos y lenguaje natural para entender objetivos de alto nivel.
2. La Orquestación para Decidir: Determina el "siguiente paso" lógico basándose en el estado y la memoria.
3. Las Herramientas (MCP) para Transformar: Actúa directamente sobre el entorno físico mediante protocolos como MQTT o Modbus, cerrando el ciclo entre el pensamiento digital y la acción mecánica.

En conclusión, la integración total de la memoria permite que el agente pase de ser un consultor pasivo a un operador dinámico capaz de gestionar la complejidad de la Industria 4.0, asegurando que cada acción física sea el resultado de un análisis histórico y situacional profundo.

1.3. LangChain

Dentro de la arquitectura de tres capas, LangChain se posiciona como el conjunto de herramientas (toolkit) fundamental para construir flujos de trabajo avanzados con modelos de lenguaje, operando específicamente como el motor interno de la capa de orquestación. Su función principal es servir como el "cableado" lógico que permite integrar y gestionar componentes críticos como prompts, modelos, herramientas y técnicas de RAG (Generación Aumentada por Recuperación).

Al utilizar el lenguaje de expresión de LangChain (LCEL), el sistema trasciende las aplicaciones lineales para convertirse en una estructura capaz de decidir "qué hacer a continuación" basándose en el contexto y las reglas de negocio establecidas. En el ámbito de la Industria 4.0, LangChain permite "cablear" la inteligencia lógica con la acción física, facilitando que el agente utilice su memoria para analizar datos históricos y razonar antes de ejecutar cualquier operación sobre el hardware. Así, LangChain transforma a la IA en un operador dinámico que no solo genera texto, sino que orquesta acciones precisas que transforman el entorno productivo de la planta.

1.4. LangGraph

LangGraph es una librería construida sobre los bloques de LangChain que permite definir flujos de trabajo mediante grafos. Se sitúa jerárquicamente por encima de LangChain, utilizando sus componentes (como modelos y herramientas) para orquestar comportamientos más sofisticados.

Conceptos Clave de Operación

Para implementar una orquestación robusta, debe dominar los siguientes pilares de LangGraph:

1. **Nodos y Aristas (Nodes & Edges):** El flujo se define como un grafo donde los nodos representan funciones de procesamiento y las aristas dictan el camino hacia el siguiente paso.
2. **Estado (State):** LangGraph mantiene un objeto de estado compartido que persiste a través de los nodos. En la industria, esto permite que el agente "recuerde" lecturas de sensores previas durante un ciclo de decisión.
3. **Ciclos y Reintentos (Retries):** A diferencia de las cadenas simples, permite bucles de retroalimentación. Si un sensor reporta un error de lectura, el grafo puede reintentar la conexión o ejecutar una rutina de limpieza automáticamente.
4. **Human-in-the-loop (HITL):** Es la capacidad de pausar la ejecución del agente para esperar la aprobación o corrección de un operador humano antes de ejecutar una acción física crítica en la planta.

1.4.1. LangGraph como el "Cerebro" de la Industria 4.0

En el contexto, LangGraph es lo que permite que el sistema deje de ser un "ChatGPT wrapper" (un espejo estático de texto) y se convierta en un operador dinámico.

1. En el nivel superior de la lógica se encuentra LangGraph, que actúa como el "sistema nervioso central" de la aplicación. Su función es definir el grafo de control, el estado y el flujo de trabajo del agente.

Gestión de Estados: A diferencia de un chat convencional, LangGraph mantiene la memoria y el estado del proceso, permitiendo ciclos de intento y persistencia de datos industriales.

Toma de Decisiones: Es aquí donde el sistema decide "qué hacer a continuación" basándose en el contexto operativo y las reglas de negocio predefinidas.

Intervención Humana: Permite integrar bucles de "Human-in-the-loop" (HITL), pausando acciones críticas en la planta para esperar la aprobación de un operador.

2. Capa de Ejecución (LangChain) LangGraph se apoya en los bloques de construcción de LangChain para ejecutar tareas específicas dentro de la orquestación. LangChain funciona como un "toolkit" que gestiona:

Componentes: Manejo de prompts, modelos de lenguaje (LLMs) y técnicas de RAG (Generación Aumentada por Recuperación) para consultar manuales técnicos o históricos.

LCEL: Utiliza el lenguaje de expresión de LangChain para "cablear" de forma eficiente la lógica con las herramientas de hardware.

3. Capa de Percepción y Acción (Models & Tools) En la base de la pirámide se encuentran los Modelos, Bases de Datos y herramientas de Hardware. Esta capa permite que la IA deje de ser un "wrapper" estático y se convierta en un operador dinámico:

Percepción: A través de protocolos como MCP, MQTT u OPC-UA, el agente recibe telemetría en tiempo real desde los sensores de la planta.

Acción: El agente utiliza herramientas para transformar el entorno físico, ejecutando comandos sobre PLCs o actuadores.

Memoria e Inteligencia: Combina el razonamiento del LLM con la memoria de largo plazo almacenada en bases de datos para realizar mantenimiento predictivo.

4. En conclusión, el sistema funciona como un ciclo cerrado donde la UI (paneles o chats) es solo el reflejo de un proceso de razonamiento autónomo orquestado por LangGraph, ejecutado por LangChain y materializado a través de herramientas industriales conectadas.

1.4.2. Ingeniería de Prompts: Del Texto a la Estructura de Datos

Tratar la salida del LLM como datos, no como párrafos Cuando usas ChatGPT como usuario, esperas una conversación. Pero cuando eres desarrollador, los párrafos son un problema porque contienen "ruido" (palabras de relleno, cortesía, explicaciones innecesarias).

1. "Trata la salida del LLM como datos, no como párrafos"
 - Normalmente, usamos modelos de lenguaje (LLMs) para escribir correos o ensayos. Sin embargo, en el desarrollo de software moderno, queremos que la IA sea un componente de nuestra aplicación.
 - El problema: El texto libre (párrafos) es difícil de procesar para un programa.
 - La solución: Pedirle a la IA que responda estrictamente en formato JSON. Esto permite que el sistema "entienda" la respuesta automáticamente sin necesidad de que un humano la lea.
2. "JSON = predecible, testeable e integrable" ¿Por qué JSON? Porque es el lenguaje universal de la web.
 - Predecible:: Sabes exactamente dónde estará cada dato (ej. el precio estará siempre en la clave "precio").
 - Testeable: Puedes escribir pruebas automáticas para verificar si la IA está respondiendo correctamente.
 - Integrable: Los datos pueden pasar directamente a una base de datos o a una interfaz de usuario sin pasos intermedios complicados.
3. Aplicar estandar Zod
 - Este es el punto más técnico y crítico para la estabilidad del sistema. Zod es una librería de validación de esquemas que actúa como un contrato.
 - El Riesgo: Las IAs a veces "alucinan" o fallan en el formato (por ejemplo, ponen un texto donde debería ir un número).
 - La Solución (Zod): Defines un contrato estricto. Si la IA devuelve algo que no encaja exactamente con la "forma" definida (por ejemplo, falta un campo obligatorio), Zod lanza un error inmediatamente.
 - Resultado: Tu aplicación nunca procesará datos corruptos o malformados, lo que evita errores inesperados en producción.

1.5. Tokens

Dado que no siempre se puede contar tokens exactos a ojo, se utiliza una aproximación estadística para el lenguaje humano ver 1.1.

$$Tokens \approx \frac{Palabras}{0,75} \quad (1.1)$$

1. Aplicación: Si tienes un documento de 1,500 palabras, puedes estimar que consumirá unos 2,000 tokens.
2. Nota técnica: Para código de programación (como el que usas en Node.js o React), la relación cambia porque los espacios y símbolos cuentan más; ahí la relación suele ser casi 1 : 1.

Para presupuestar una aplicación, se calcula el costo por cada mil tokens (MTok) ver 1.2.

$$Costo_{total} = (Tokens_{entrada} \times Tarifa_{input}) + (Tokens_{salida} \times Tarifa_{output}) \quad (1.2)$$

Dato clave: Casi todos los modelos (como GPT-4 o Claude) cobran más caro por los tokens que la IA genera (output) que por los que tú le envías (input).

3. Límite de la Ventana de Contexto

La ventana de contexto es un recurso finito. El cálculo que debes hacer para que tu aplicación no "falle" es ver 1.3:

$$Contexto_{libre} = Ventana_{máxima} - (Tokens_{instrucciones} + Tokens_{datos} + Tokens_{historial}) \quad (1.3)$$

Truncamiento: Si el resultado es negativo, el modelo truncará la información y perderá coherencia.

Optimización: Aquí es donde entran tus proyectos de agentes de IA; un buen agente debe decidir qué información "olvidar" o resumir para no saturar este cálculo.

El término Temperature (Temperatura)

Es un hiperparámetro crucial que controla el grado de aleatoriedad o "creatividad" de las respuestas de un modelo de lenguaje. En términos técnicos, la temperatura ajusta las probabilidades de las palabras (tokens) que el modelo considera para su siguiente respuesta. No cambia el conocimiento de la IA, sino qué tan "arriesgada" es al elegir la siguiente palabra.

- Valores bajos (0.1 - 0.3): El modelo se vuelve determinista. Elige siempre la palabra más probable. Es ideal para tareas que requieren precisión, como escribir código, resolver problemas matemáticos o extraer datos en formato JSON.
- Valores altos (0.7 - 1.2+): El modelo se vuelve creativo y diverso. Empieza a elegir palabras menos obvias, lo que genera respuestas más variadas, poéticas o sorprendentes. Es ideal para lluvia de ideas o escritura creativa.

El Cálculo Matemático Detrás

Para entender cómo soporta esto la información de tus diapositivas anteriores, la temperatura actúa sobre una función llamada Softmax. Si el modelo tiene varias opciones para la siguiente palabra con sus respectivas "puntuaciones" (z_i), la fórmula de probabilidad P_i ajustada por la temperatura (T) es ver 1.4:

$$P_i = \frac{\exp(z_i/T)}{\sum (z_j/T)} \quad (1.4)$$

- Si T es alta: Las diferencias entre las probabilidades se "aplanan", haciendo que palabras menos probables tengan una oportunidad real de ser elegidas.
- Si T es baja: Las diferencias se exageran, haciendo que la opción más probable domine por completo.

Dado que se está trabajando en agentes industriales y sistemas agénticos que deben usar herramientas y razonar, la configuración de la temperatura es vital:

1. Para el Agente de Mantenimiento o PLCs: Deberías usar una temperatura cercana a 0. No quieres que la IA "sea creativa" al decidir si una máquina tiene un fallo en Factory I/O; necesitas una respuesta lógica y predecible basada en datos.
2. Para el Agente con o un agente de respuesta conversacional: Podrías usar una temperatura media (0,7) para que la conversación se sienta natural, fluida y no siempre responda lo mismo, dándole "personalidad" al agente.

La eficiencia de un sistema de IA no solo depende de los datos, sino de la gestión inteligente del contexto. Cada interacción suma información proveniente de diversas fuentes: las instrucciones del sistema, la entrada del usuario, el historial de la conversación y las respuestas de las herramientas externas. Mantener este flujo corto y conciso es fundamental, ya que el aumento de tokens no solo incrementa el costo financiero, sino que eleva la latencia, haciendo que el agente sea más lento al procesar y generar respuestas.

Anatomía del Contexto (Lo que el modelo "lee")

Para los proyectos de agentes industriales y aplicaciones móviles, debes considerar que la "ventana de contexto" se llena con:

- System Instructions: Las reglas base que definen el comportamiento del agente.
- User Input: Lo que el usuario escribe o solicita en el momento.
- Chat History: Los mensajes anteriores que permiten que la IA tenga memoria de la conversación.

- **Tools Responses:** Los datos crudos que regresan tus herramientas (por ejemplo, el estado de un sensor o el log de un PLC).
- **Its own output:** La respuesta que la IA acaba de generar también ocupa espacio para la siguiente interacción.

Impacto en el Rendimiento y Costo

Hay una relación directa entre volumen y velocidad:

1. Más tokens = Más que leer: Aumenta el tiempo que el modelo tarda en "entender" el contexto antes de empezar a responder.
2. Más tokens de salida = Más que generar: La generación de texto es la parte más lenta del proceso. A mayor longitud, mayor será la latencia (el tiempo de espera del usuario).
3. Latencia y Costo: Son los dos grandes enemigos. Un contexto saturado hace que tu aplicación se sienta pesada y que el consumo de la API se dispare innecesariamente.

Regla de Oro para el Desarrollador

"Short and concise > long, random dumping" (Corto y conciso es mejor que volcar información larga y aleatoria). En lugar de enviarle a la IA todos los datos históricos de un sensor, es mejor enviarle solo un resumen o los últimos 5 minutos de lectura. Esto garantiza que el agente sea rápido, preciso y económico.

Ajuste de Precisión: Temperatura y Parámetros

Aquí es donde configuras "la personalidad" de tu agente:

- **Temperature (Estabilidad vs. Creatividad):**
 - Baja (0.1 - 0.2): Respuestas lógicas y consistentes. Ideal para tus PLCs y agentes industriales.
 - Alta (0.8+): Respuestas variadas. Ideal para el Tamagotchi o conversaciones fluidas.
- **Top_P:** Controla qué tan amplio es el abanico de palabras que la IA considera. Ayuda a filtrar opciones irrelevantes.
- **Max_Tokens:** Pone un límite estricto al largo de la respuesta para ahorrar dinero y evitar que la IA hable de más.
- **Penalties (Penalizaciones):** Evitan que la IA se vuelva repetitiva y use las mismas palabras una y otra vez.

1.6. Modelos de Chat vs. Herramientas (Tool/JSON)

No todos los modelos de IA deben usarse de la misma manera. La elección depende de si necesitas una conversación humana o una ejecución técnica precisa ver 1.2.

1. Chat models: free-form answers

Los modelos de chat están optimizados para la fluidez y la naturalidad.

Uso: Respuestas en formato de texto libre (párrafos).

Ideal para: Tu proyecto de asesoramiento, donde lo importante es la personalidad y la variación en el lenguaje.

2. Modelos de Herramientas (Tools/JSON)

Estos modelos están entrenados específicamente para seguir reglas estructurales estrictas.

Uso: Generan código, llaman a funciones (functions calling) o devuelven datos en JSON.

Ideal para: Tus proyectos de PLCs, sensores y agentes de mantenimiento. Aquí no necesitas que la IA sea "simpática", sino que te entregue un dato exacto que tu aplicación pueda procesar sin errores.

3. Elige según la tarea, no por la moda: Es una advertencia sobre la ingeniería eficiente. No siempre el modelo más grande o más famoso es el mejor para tu tarea específica.

Eficiencia: Para leer un sensor o detectar un fallo, un modelo pequeño y especializado en JSON (como Gemini Flash) será mucho más rápido y económico que un modelo gigante de chat generalista.

Los modelos mostrados en la tabla son ideales para tus sistemas agénticos industriales, donde la IA debe razonar, decidir el siguiente paso y usar herramientas.

Estos modelos son ideales para tus sistemas agénticos industriales, donde la IA debe razonar, decidir el siguiente paso y usar herramientas.

Categoría	Modelo Recomendado	Ejecución	Fortaleza	Tarea Ideal
Agéntico / Potencia	Gemini 2.5 Pro	Nube	Ventana de contexto masiva para leer manuales extensos de PLCs	Orquestación de toda la planta y toma de decisiones complejas.
Costo-Eficiente	GPT-5 mini / Gemini 2.5 Flash	Nube	Alta velocidad y costo extremadamente bajo por token	Procesamiento de formularios de entrada y reportes rápidos de sensores.
Edge (Local)	Phi-4 / Llama 3.2 (3B)	Local (Raspberry Pi)	Muy ligero; corre sin internet en tus dispositivos maestros/esclavos	Control en tiempo real de sensores y lógica de PLC Industrial Shields
Visión	Qwen2.5-VL	Nube/Híbrido	Capacidad avanzada para entender mapas y coordenadas satelitales.	Conteo de plántulas y localización de árboles mediante imágenes.
Razonamiento Técnico	DeepSeek V3	Nube	Excelente en matemáticas y lógica de programación estructurada.	Generación de scripts de automatización y depuración de errores en Factory I/O.

Tabla 1.2: Modelos Agénticos (Para toma de decisiones y orquestación).

1.7. Zod?

Zod es una librería de declaración y validación de esquemas para TypeScript y Node.js. En el contexto de los LLMs, se utiliza para definir un "contrato" de datos. Si la IA es el motor que genera información, Zod es el inspector de calidad que asegura que esa información tenga la forma exacta que tu código espera.

1. ¿Cómo funciona en tu modelo de Agente? Cuando trabajas con agentes que controlan hardware (como tus PLCs o sensores NPK), no puedes permitir que la IA responda con un párrafo amistoso. Necesitas datos. El flujo funciona así:
 - a) Definición del Esquema: Tú defines qué datos necesitas (ej. un número para el pH, un booleano para una válvula).
 - b) Prompting Estructurado: Le pasas ese esquema a la IA (usualmente convertido a JSON Schema) diciéndole: "Solo puedes responder siguiendo esta estructura".
 - c) Validación en Tiempo Real: Cuando el modelo responde, Zod toma esa respuesta y la analiza.
 - Si es correcta: Los datos pasan a tus funciones de control.
 - Si es incorrecta: Zod detiene el proceso y te da un error detallado (o le pide a la IA que corrija el error)

2. Para la Industria 4.0, la integración de esquemas de validación como Zod no es solo una buena práctica de programación, sino una necesidad de ingeniería para garantizar que la Inteligencia Artificial sea operativa en entornos reales.

Concepto de "contrato de datos" es fundamental de manera general:

3. Garantía de Seguridad Operativa

- a) En un entorno industrial, las palabras de un modelo de lenguaje deben transformarse en señales para el mundo físico.
 - Prevención de comandos erróneos: Evita que una respuesta ambigua de la IA se convierta en una instrucción peligrosa para una máquina o un actuador.
 - Integridad del sistema: Al imponer una estructura rígida, aseguras que los motores, sensores y controladores lógicos solo reciban parámetros que estén dentro de los rangos seguros definidos previamente.

4. Robustez Técnica y Tipado

- Independientemente del lenguaje o framework que utilices, la validación de esquemas aporta una capa de certeza técnica:
 - a) Seguridad de tipos: El sistema conoce exactamente qué tipo de dato debe recibir (número, booleano, cadena), permitiendo que el compilador detecte errores antes de que el código se ejecute.
 - b) Mantenimiento simplificado: Si el esquema del proceso industrial cambia, solo necesitas actualizar la definición en un lugar; el sistema te alertará automáticamente en todos los puntos donde esa estructura se vea afectada.

5. Mitigación de "Alucinaciones" en Datos

- Los modelos de IA pueden generar información falsa o campos inexistentes por su naturaleza probabilística.
 - a) Filtro de precisión: Actúa como una aduana que bloquea cualquier dato que no cumpla con el formato exigido.
 - b) Consistencia de datos: Garantiza que, si pides el estado de un sensor, recibas un dato procesable y no una explicación textual innecesaria.

6. El Esquema como Fuente Única de Verdad Un contrato de tiempo de ejecución unifica la intención del programador con el comportamiento de la IA:

- Sin duplicidad de lógica: No hace falta programar validaciones manuales complejas; el esquema define las reglas y las aplica en el momento en que llega la información.

- **Confiabilidad:** Transforma una salida de IA "creativa" en un componente de software predecible, permitiendo que cualquier aplicación consuma estos datos con total confianza en su formato y contenido.

Aplicación para tu agente de mantenimiento industrial, esta estructura significa que:

- Defines un contrato con Zod para los fallos del PLC.
- Usas un Tool Model para que la IA elija qué herramienta de reparación usar.
- Controlas los Tokens para que el historial no sature la memoria del proceso.

1.8. Estructura de salida

Es la técnica para obligar a un modelo de IA a responder con un formato de datos estricto en lugar de texto libre.

1. Pedir al modelo que complete un esquema, no que adivine el formato: En lugar de darle libertad creativa, le entregas un molde o "esquema" predefinido (como los esquemas de Zod vistos anteriormente) para que la IA solo rellene los huecos con la información correspondiente.
2. Uso de funciones nativas del proveedor: Para lograr esta precisión, se utilizan herramientas integradas de los modelos como el Modo JSON o el Tool Calling (llamada a funciones). Estas funciones internas aseguran que la respuesta cumpla con las reglas sintácticas del formato de datos elegido.
3. Mayor fiabilidad que un simple "por favor, responde en JSON": Esta es la mejora técnica más importante. Simplemente pedirle a una IA que use JSON en un mensaje de chat suele fallar (la IA puede añadir texto extra o cometer errores de formato). Al usar Salidas Estructuradas, el sistema garantiza que el resultado sea siempre un objeto de datos válido que tu aplicación puede procesar de inmediato sin romperse.

El Esquema (El "Molde")

1. Primero, se define la estructura que necesitas usando un contrato como Zod ver 1.1:

Algoritmo 1.1 Esquema de validación Zod.

```
TypeScript
const InformeMotorSchema = z.object({
  id_motor: z.string(),
  temperatura_celsius: z.number(),
  estado: z.enum(["operativo", "mantenimiento_requerido", "critico"]),
  piezas_a_revisar: z.array(z.string()),
  confianza_diagnostico: z.number().min(0).max(1) });
```

2. La Entrada (Input) Se envía a la IA generalmente un texto desordenado o un log de sensores:

"El motor MTR-204 está operando a 85 grados. Se nota una vibración inusual en los rodamientos y la banda de distribución parece gastada. El técnico sugiere revisión inmediata."

3. La Salida Estructurada (Output) debido a la configuración 1.1 el modelo en Modo JSON o Tool Calling, la IA ignora las reglas de cortesía y devuelve estrictamente esto ver 1.2:

Algoritmo 1.2 Respuesta del modelo.

```
Json
{
  "id_motor": "MTR-204",
  "temperatura_celsius": 85,
  "estado": "mantenimiento_requerido",
  "piezas_a_revisar": ["rodamientos", "banda de distribución"],
  "confianza_diagnostico": 0.95
}
```

¿Por qué esto es mejor que el chat normal? Sin "adivinanzas": El modelo no intenta adivinar el formato; rellena el esquema que tú le diste.

- Integración inmediata: Tu aplicación en Node.js o React puede leer ese objeto JSON y encender una alerta roja automáticamente sin que un humano intervenga.
- Fiabilidad total: Si la IA intenta enviar algo que no sea un número en temperatura_celsius, el sistema (vía Zod) lo rechaza antes de que cause un error en tus bases de datos

Capítulo 2

Servidor

El servidor constituye el componente central del sistema y actúa como el núcleo encargado de coordinar la comunicación, el procesamiento de datos y la integración de servicios. Su función principal es recibir las solicitudes provenientes de los clientes a través de la API REST, gestionar la lógica de negocio mediante controladores y servicios especializados, y garantizar la interacción con los distintos subsistemas como la base de datos, los servicios de comunicación en tiempo real y los módulos de inteligencia artificial. Además, el servidor mantiene conexiones activas con tecnologías como MQTT y WebSockets para permitir el manejo de datos en tiempo real, lo que resulta fundamental en entornos IoT y aplicaciones de monitoreo continuo. En conjunto, este componente opera como el orquestador general del sistema, asegurando la correcta ejecución de los procesos, la disponibilidad de los recursos y la coherencia en el flujo de información entre todas las capas de la arquitectura ver 2.1.

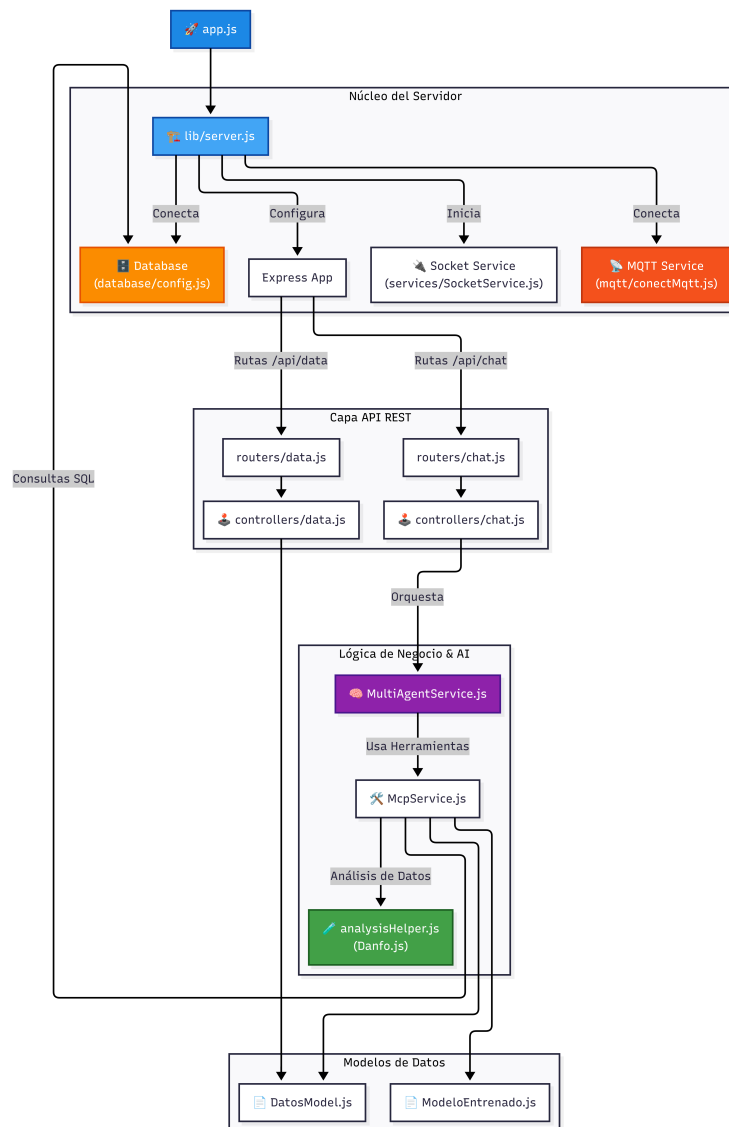


Figura 2.1: Servidor

Las variables de entorno son parámetros de configuración externos al código que permiten definir valores clave para el funcionamiento de una aplicación sin necesidad de modificarlos directamente en el programa. Se utilizan para almacenar información sensible y configurable, como puertos de ejecución, credenciales de servicios de inteligencia artificial, direcciones de bases de datos, datos de conexión a brokers MQTT, rutas de certificados de seguridad y parámetros de

servicios locales como modelos de IA. Su principal ventaja es que facilitan la portabilidad, la seguridad y la flexibilidad del sistema, ya que permiten adaptar la aplicación a distintos entornos (desarrollo, pruebas o producción) simplemente cambiando estos valores, sin alterar la lógica del software ver.

Algoritmo 2.1 variables de entorno

2.1

```
# Puerto del servidor Express (Debe coincidir con VITE_API_URL en el frontend)
PORT=8080
# Clave de OpenAI para servicios de IA
OPENAI_API_KEY=TU_API_KEY
GOOGLE_API_KEY=TU_GOOGLE_KEY
# Configuración del Broker MQTT
BROKER=mqtt://localhost:1883
MQTT_USER=plcuser
MQTT_PASS=plc
# Configuración OPC UA
SERVER_IP=192.168.20.23
OPC_ENDPOINT=opc.tcp://:4334/Plc/PlcOpcServer
OPC_HOST= OPC_PORT=4334
OPC_PATH=/Plc/PlcOpcServer
# Base de Datos Neon PostgreSQL
PG_HOST=ep-aged-sea-adtvsspi-pooler.c-2.us-east-1.aws.neon.tech
PG_PORT=5432
PG_USER=neondb_owner
PG_PASSWORD=*****
PG_DBNAME=neondb
# Configuración Ollama Local
OPENAI_BASE_URL=http://localhost:11434/v1
OPENAI_MODEL=gpt-4o
SSL_ENABLED=true
TLS_KEY_PATH=./certs/server.key
TLS_CERT_PATH=./certs/server.crt
TLS_CA_PATH=./certs/ca.crt
MQTT_REJECT_UNAUTHORIZED=false
```

2.1. Capa de Inicio — app.js

Esta es la primera capa del sistema y funciona como el punto de arranque de toda la aplicación. Su responsabilidad principal es iniciar el entorno del servidor, cargar configuraciones básicas como variables de entorno y definir el puerto en

el que se ejecutará la aplicación. Además, delega la inicialización del sistema al módulo central del servidor. En términos simples, esta capa actúa como el “interruptor de encendido” del backend ver2.2.

Algoritmo 2.2 Arranque del sistema

ALGORITMO InicializarAppJS

INICIO

```
// Paso 1: Configurar entorno
    CargarModulo(dotenv)
    CargarModulo(dotenv-expand)
// Paso 2: Leer archivo .env
    variables Ejecutar(dotenv.config)
// Paso 3: Expandir variables
    Ejecutar(dotenvExpand.expand, variables)
// Paso 4: Importar servidor principal
    ServerClase ImportarModulo("lib/server")
// Paso 5: Crear instancia del servidor
    servidor CrearInstancia(ServerClase)
// Paso 6: Iniciar servidor
    EjecutarMetodo(servidor.listen)
```

FIN

2.2. Núcleo del Servidor — server.js

Esta capa representa el corazón del sistema backend. Aquí se configura la aplicación principal, se establecen las conexiones con la base de datos, se inician los servicios de comunicación como MQTT y WebSockets, y se preparan todos los módulos necesarios para que el servidor funcione correctamente. Su función principal es coordinar y centralizar el arranque de todos los servicios 2.3.

Algoritmo 2.3 Sistema Servidor

```
ALGORITMO SistemaServidor
INICIO
// Capa 1: Inicio del Sistema
    CargarVariablesEntorno()
    puerto ObtenerPuerto()
// Capa 2: Núcleo del Servidor
    servidor CrearServidor()
    ConfigurarExpress(servidor)
    ConectarBaseDatos()
    InicializarMQTT()
    InicializarWebSockets()
    RegistrarRutas()
    IniciarEscucha(puerto)
FIN
```

2.3. Capa de Comunicación — Express (API REST)

Esta capa permite la interacción entre los clientes y el servidor mediante solicitudes HTTP. Se encarga de recibir peticiones, dirigirlas a las rutas correspondientes y devolver respuestas estructuradas. Es básicamente la puerta de entrada del sistema, ya que gestiona toda la comunicación externa con aplicaciones web, móviles o servicios externos 2.4.

Algoritmo 2.4 Comunicación HTTP

```
PROCEDIMIENTO ManejarSolicitudHTTP(solicitud)
INICIO
    ruta IdentificarRuta(solicitud)
    ValidarFormato(solicitud)
    router SeleccionarRouter(ruta)
    EnviarASiguienteCapa(router, solicitud)
FIN
```

2.4. Capa de Enrutamiento — Routers

Los routers organizan las rutas del sistema según su funcionalidad. Se encargan de dirigir cada solicitud hacia el controlador adecuado dependiendo del tipo

de operación solicitada, como consultas de datos o interacciones con inteligencia artificial. Esta capa permite mantener una estructura ordenada y modular en la API ver 2.5.

Algoritmo 2.5 Enrutamiento

```
PROCEDIMIENTO RouterProcesarSolicitud(solicitud)
INICIO
    tipoOperacion DeterminarTipo(solicitud)
    SI tipoOperacion = "CHAT" ENTONCES
        controlador ChatController
    SINO
        controlador DataController
    FIN SI
    EnviarASiguienteCapa(controlador, solicitud)
FIN
```

2.5. Capa de Control — Controllers

Los controladores actúan como intermediarios entre las solicitudes de los usuarios y la lógica interna del sistema. Procesan las peticiones, validan los datos recibidos y coordinan la ejecución de servicios internos. Su objetivo es transformar una solicitud en una acción concreta dentro del sistema ver 2.6.

Algoritmo 2.6 Controladores

```
PROCEDIMIENTO ControllerEjecutar(solicitud)
INICIO
    ValidarParametros(solicitud)
    servicio DeterminarServicio(solicitud)
    resultado EjecutarServicio(servicio, solicitud)
    ConstruirRespuesta(resultado)
    EnviarRespuestaCliente()
FIN
```

2.6. Capa de Lógica de Negocio — Servicios

En esta capa se ejecutan las operaciones principales del sistema. Aquí se encuentran los servicios que manejan procesos como la inteligencia artificial

multi-agente, el análisis de datos y la interacción con herramientas especializadas. Esta capa es donde realmente se toman decisiones y se ejecutan los procesos complejos ver 2.7.

Algoritmo 2.7 Lógica de Negocio

```
PROCEDIMIENTO EjecutarServicio(servicio , solicitud)
INICIO
    SI servicio = "IA" ENTONCES
        resultado MultiAgentProcesar(solicitud)
    SINO SI servicio = "DATOS" ENTONCES
        resultado MCPProcesarConsulta(solicitud)
    FIN SI
    RETORNAR resultado
FIN
```

2.7. Capa de Herramientas — MCP Service y Helpers

Esta capa proporciona funcionalidades específicas que los servicios necesitan para operar, como ejecutar consultas SQL, procesar datos estadísticos o manipular información en estructuras avanzadas. Actúa como un conjunto de utilidades que amplían las capacidades del sistema ver 2.8.

Algoritmo 2.8 Herramientas MCP

```
PROCEDIMIENTO MCPProcesarConsulta(solicitud)
INICIO
    tipoHerramienta IdentificarHerramienta(solicitud)
    SI tipoHerramienta = "SQL" ENTONCES
        datos EjecutarConsultaSQL(solicitud)
    SINO SI tipoHerramienta = "ANALISIS" ENTONCES
        datos EjecutarAnalisisDatos(solicitud)
    FIN SI
    RETORNAR datos
FIN
```

2.8. Capa de Persistencia — Base de Datos y Modelos

Es la capa encargada de almacenar y recuperar la información del sistema. Aquí se definen los modelos de datos, las estructuras de almacenamiento y las conexiones con el motor de base de datos. Garantiza la integridad y disponibilidad de la información ver 2.9.

Algoritmo 2.9 Persistencia de Datos

```
PROCEDIMIENTO EjecutarConsultaSQL(solicitud)
INICIO
    conexion ObtenerConexionBD()
    consulta TraducirConsulta(solicitud)
    resultado EjecutarEnBaseDatos(conexion, consulta)
RETORNAR resultado
FIN
```

2.9. Capa de Comunicación en Tiempo Real — MQTT y WebSockets

Esta capa permite el intercambio continuo de datos en tiempo real. MQTT facilita la comunicación con dispositivos IoT, mientras que los WebSockets permiten enviar información instantánea a los clientes conectados. Es esencial para sistemas de monitoreo y telemetría ver 2.10.

Algoritmo 2.10 Comunicación en Tiempo Real

```
PROCEDIMIENTO GestionarTiempoReal()
INICIO
    ConectarBrokerMQTT()
    SuscribirseTemas()
    MIENTRAS sistemaActivo HACER
        mensaje RecibirMensajeMQTT()
        ProcesarDatos(mensaje)
        EmitirEventoWebSocket(mensaje)
    FIN MIENTRAS
FIN
```

Capítulo 3

Agentes

3.1. Modelos de IA locales

La diferencia fundamental entre un modelo local y uno en línea reside en la soberanía de los datos y la latencia. Mientras que un modelo en línea (como GPT-4 o Claude) depende de servidores externos, conexión a internet y políticas de privacidad de terceros, un modelo local se ejecuta íntegramente en tu hardware, garantizando una privacidad absoluta y permitiendo el funcionamiento sin conexión. Sin embargo, el modelo local está limitado por la potencia de tu GPU (VRAM), mientras que los modelos en la nube aprovechan infraestructuras masivas de centros de datos para ofrecer capacidades de razonamiento superiores a costa de una mayor latencia de red y posibles costos por suscripción o uso de API.

Guía de Instalación de Modelos Locales Para correr modelos de lenguaje (LLMs) en tu propia máquina de forma sencilla, la herramienta estándar actualmente es Ollama.

1. Instalación del Entorno Descarga: Ve a www.ollama.com descarga la versión para tu sistema operativo (Windows, macOS o Linux).

Ejecución: Una vez instalado, abre una terminal (CMD en Windows o Terminal en Mac/Linux).

2. Descarga de Modelos Existen diferentes tamaños según tu hardware. Los más comunes son:

Llama 3 (8B): Necesitas al menos 8GB de RAM/VRAM.

Mistral: Muy eficiente y balanceado.

DeepSeek-Coder: Excelente para el desarrollo de software y circuitos.

3. Para instalar uno, escribe en tu terminal:

Bash:

```
ollama run llama3
```

4. Configuración para un Rendimiento Óptimo para que el modelo corra de forma fluida, debes considerar lo siguiente:

Prioridad de GPU: Asegúrate de tener instalados los drivers de NVIDIA (CUDA). Ollama detectará automáticamente tu tarjeta de video para acelerar el proceso.

Cuantización: Los modelos vienen en versiones "comprimidas" (ej. Q4_K_M). Esto permite que un modelo grande quepa en una tarjeta de video comercial sin perder mucha precisión.

Context Window: Si vas a pasarle esquemáticos largos o mucho código, asegúrate de configurar el límite de memoria en la aplicación que uses como interfaz.
5. El servicio de Ollama permiten mejorar el rendimiento cuando se trabajan modelos de inteligencia artificial locales, ya que habilitan la ejecución simultánea y el mantenimiento de varios modelos cargados en memoria. La variable `OLLAMA_NUM_PARALLEL=2` define cuántas solicitudes pueden procesarse al mismo tiempo, lo que es especialmente útil en sistemas con múltiples agentes o aplicaciones que realizan consultas concurrentes. Por su parte, `OLLAMA_MAX_LOADED_MODELS=2` indica cuántos modelos pueden permanecer cargados en la RAM sin tener que descargarse y recargarse constantemente, reduciendo significativamente los tiempos de respuesta. En conjunto, estas opciones optimizan el uso de recursos del sistema y permiten ejecutar, por ejemplo, un modelo para análisis de datos y otro para generación de código de manera paralela, algo clave en arquitecturas modernas de backend con IA local, ver 3.1.

Algoritmo 3.1 Configuración de modelos locales

```
sudo systemctl edit ollama.service
#Copia y pega
[Service]
Environment="OLLAMA_NUM_PARALLEL=2"
Environment="OLLAMA_MAX_LOADED_MODELS=2"
#Guarda y restaurar el sistema
sudo systemctl daemon-reload
sudo systemctl restart ollama
#Para probar se debe correr en terminales distintos
ollama run llama3.1
ollama run deepseek-coder:6.7b
```

3.2. Servicio Multi agente

El 3.1diagrama de secuencia describe el flujo completo de interacción entre el usuario y el sistema de agentes inteligentes dentro de la arquitectura del proyecto. El proceso inicia cuando el usuario, desde el frontend, envía una solicitud al endpoint `/api/chat`, incluyendo no solo el mensaje de texto sino también el contexto de la interfaz, como identificadores seleccionados o información adicional proveniente de la aplicación. Esta solicitud es recibida por el Router, el cual se encarga de direccionarla hacia el ChatController, componente responsable de validar los datos de entrada, estructurar el contexto del sistema e integrar elementos relevantes provenientes de la interfaz de usuario.

Posteriormente, el controlador envía la consulta al MultiAgentService, que constituye el núcleo inteligente del sistema. En esta etapa se construye dinámicamente un grafo de agentes utilizando LangGraph, permitiendo que el sistema determine de manera autónoma qué tipo de agente debe responder según la intención de la consulta. Para ello, el servicio consulta al modelo de lenguaje, que actúa como orquestador, evaluando si la pregunta requiere acceso a datos, análisis estadístico o una respuesta general. Una vez identificada la intención, el sistema activa el agente correspondiente, el cual utiliza el modelo de inteligencia artificial para generar una respuesta contextualizada y, si es necesario, procesar información adicional.

Finalmente, el servicio retorna al controlador una respuesta estructurada que puede incluir texto explicativo y datos procesados. El controlador envía esta información al usuario en formato JSON, permitiendo que el frontend presente resultados en tiempo real, gráficos o análisis detallados. Este flujo refleja cómo el sistema integra comunicación web, procesamiento contextual, orquestación de agentes y modelos de inteligencia artificial para ofrecer respuestas inteligentes basadas tanto en la consulta del usuario como en el contexto operativo del sistema.

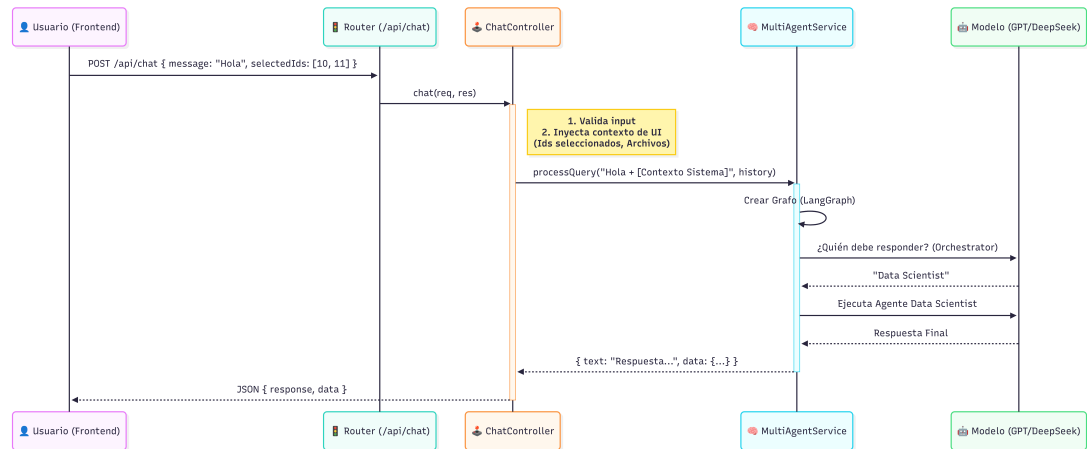


Figura 3.1: Diagrama Multi agente

La Grafica 3.2 muestra las acciones que cada uno de los bloques representa.

1. MultiAgentService (LangGraph) – Cerebro del sistema

- Este bloque, llamado Brain, es el núcleo inteligente del sistema.
 - a) Orchestrator Node: actúa como coordinador. Decide la intención de la consulta: si es simple (acceso a base de datos) o compleja (requiere análisis de datos).
 - b) SQL Expert Node: maneja consultas simples directamente sobre la base de datos.
 - c) Data Scientist Node: se activa cuando se requiere un análisis más avanzado o procesamiento de datos complejo.
 - d) La flecha SQL_Node → Data_Node representa un “handover”: si una consulta simple necesita análisis adicional, se transfiere automáticamente al agente de datos.

2. McpService – Herramientas de soporte

- Este bloque contiene las herramientas o funciones que usan los agentes:
 - a) Tool query_db: ejecuta consultas SQL sobre los modelos de datos.
 - b) Tool analizar_datos_avanzado: realiza procesamiento y análisis avanzado de los datos.

3. Capas de Datos (Models)

- Aquí se representan los modelos y scripts que gestionan la información:

- a) DatosModel.js: gestiona la tabla principal de datos (datos).
 - b) ModeloEntrenado.js: almacena y gestiona modelos de inteligencia artificial previamente entrenados.
 - c) AnalysisHelper.js (Danfo): lógica para análisis avanzado usando Danfo.js (similar a Pandas en JavaScript).
4. Base de Datos (PostgreSQL)
- Representa el almacenamiento físico de la información:
 - a) Tabla datos: contiene la información principal que consultan los agentes.
 - b) Tabla modelo_entrenado: almacena los modelos de IA entrenados que pueden ser reutilizados.
5. Flujo de información
- El Orchestrator decide qué agente debe atender la consulta.
 - a) Los agentes llaman a las herramientas según el tipo de operación (consulta simple o análisis avanzado).
 - b) Las herramientas interactúan con los Modelos (DatosModel y AnalysisHelper) y ejecutan operaciones sobre la Base de Datos.
 - c) Los datos procesados se retornan hacia los agentes y finalmente al Orchestrator, que devuelve la respuesta al sistema que hizo la solicitud.

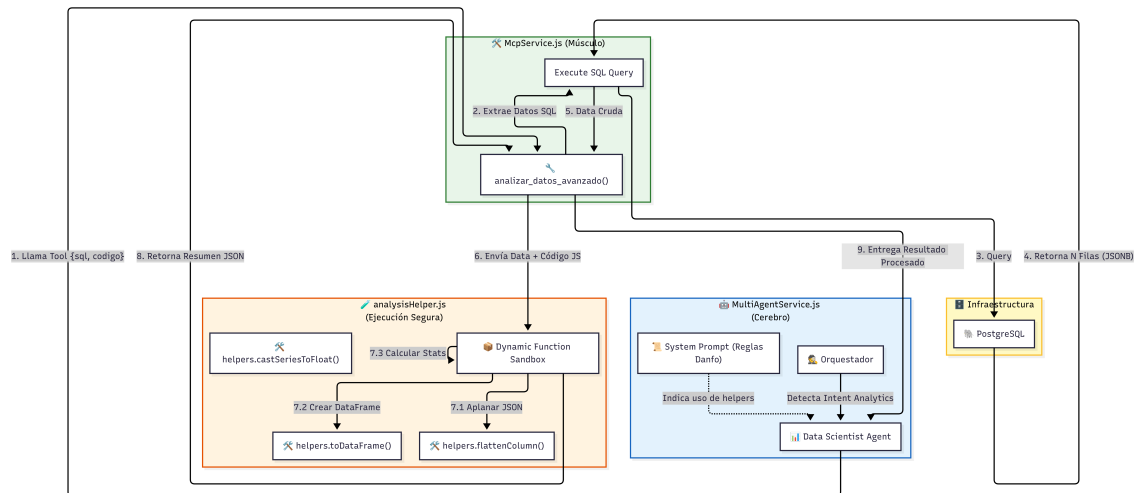


Figura 3.2: Lang Graph Orquestador

Capítulo 4

Certificado TLS, MQTTS

4.1. Guía de instalación y configuración de Mosquitto MQTT Broker

Introducción

El protocolo MQTT (Message Queuing Telemetry Transport) es un estándar ligero de mensajería basado en el modelo publicador/suscriptor, ideal para la comunicación entre dispositivos IoT y servidores o aplicaciones industriales.

El broker MQTT actúa como intermediario entre los clientes que publican mensajes (publishers) y los que los reciben (subscribers). En este caso, el broker será el PC y el cliente será el PLC.

1. Actualizar el sistema

```
sudo apt update
```

2. Instalar Mosquitto y las herramientas cliente

```
sudo apt install mosquitto mosquitto-clients -y
```

Estas herramientas instalan tanto el servidor (broker) como los comandos de prueba `mosquitto_pub` y `mosquitto_sub`.

3. Iniciar y habilitar el servicio

Habilitar el servicio para que inicie automáticamente al arrancar:

```
sudo systemctl enable mosquitto
```

Ejecución directa con salida en consola (modo verbose):

```
mosquitto -v
```

4. Verificación de funcionamiento

Para comprobar que el servicio escucha en el puerto 1883 (por defecto):

```
sudo lsof -i :1883
```


Si se desea usar otro puerto:

```
mosquitto -p 1884 -v
```

5. Permitir conexiones externas

Por defecto, Mosquitto puede estar configurado solo para conexiones locales. Para habilitar conexiones externas (por ejemplo, desde el Esp32):

```
sudo nano /etc/mosquitto/mosquitto.conf
```

```
listener 1883 0.0.0.0
protocol mqtt
listener 8083 0.0.0.0
protocol websockets
# — Seguridad
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/aclfile
# — Logs
connection_messages true
log_type all
log_dest syslog
log_dest stdout
```

Guarda con Ctrl + O, luego Enter y sal con Ctrl + X. Después, reinicia el servicio:

```
sudo systemctl restart mosquitto
```

6. Configuración del acceso seguro (usuarios y permisos)

Crear un usuario y contraseña. Ejemplo: crear usuario `plcuser` con contraseña `plc`:

```
sudo mosquitto_passwd -c /etc/mosquitto/passwd plcuser
```

Para añadir más usuarios (sin sobrescribir el archivo):

```
sudo mosquitto_passwd /etc/mosquitto/passwd otroUser
```

Crear archivo de control de acceso (ACL). Define los permisos de publicación y suscripción por tema (topic).

```
sudo nano /etc/mosquitto/aclfile
```

Contenido recomendado:

```
# — 1. CONFIGURACIÓN
user plcuser
#Estos se mantienen para la autonomía interna y la
  comunicación funcional
topic readwrite Plc/Esp32
```

Guarda los cambios y reinicia el servicio:

```
sudo systemctl restart mosquitto
```

Verifica nuevamente:

```
sudo systemctl status mosquitto sudo lsof -i :1883
```

7. Pruebas de comunicación

- Suscriptor (en el BROKER / PC)

Escucha todos los tópicos que empiecen con Plc/:

```
mosquitto_sub -h localhost -t "Plc/#" -u plcuser -P "plc"
```

- Publicador (en el CLIENTE / PLC)

Envía un mensaje de prueba:

```
mosquitto_pub -h <IP_DEL_BROKER> -t "Plc/Adc" -m  
"123" -u plcuser -P "plc"
```

Ejemplo: Si el PC tiene IP 192.168.1.100, entonces:

```
mosquitto_pub -h 192.168.1.100 -t "Plc/Adc" -m "123"  
-u plcuser -P "plc"
```

8. Recomendaciones de seguridad

Deshabilita `allow_anonymous true` una vez configuradas las contraseñas.

Usa un puerto diferente de 1883 (por ejemplo, 1884 o 1885) para entornos con múltiples brokers.

Si el PLC se conecta desde otra red, abre el puerto en el firewall:

```
sudo ufw allow 1883/tcp
```

El riesgo principal de no utilizar MQTTS (MQTT sobre TLS/SSL) radica en que los datos viajan en texto plano, lo que expone toda la comunicación de tu red a ataques de interceptación (Eavesdropping). En un entorno donde dispositivos como sensores industriales o nodos de control gestionan información crítica, un atacante en la misma red podría no solo robar credenciales de acceso, sino también realizar ataques de Man-in-the-Middle (MitM) para inyectar comandos falsos o manipular las lecturas de los sensores. Sin el cifrado que proporciona la capa TLS, la integridad y la confidencialidad del sistema quedan totalmente vulneradas, transformando un proyecto de automatización en una puerta abierta para el acceso no autorizado a tu infraestructura local.

Riesgos Críticos de usar MQTT Simple: Exposición de Credenciales: El usuario y la contraseña del broker se pueden capturar fácilmente con herramientas de rastreo de paquetes (como Wireshark).

Manipulación de Datos: Un tercero puede publicar mensajes en tus tópicos, activando actuadores o enviando alertas falsas.

Falta de Autenticidad: No hay garantía de que el dispositivo que envía la información sea realmente tu sensor y no un suplantador.

4.2. Procedimiento de configuración de TLS y WebSocket seguro

Creación de la infraestructura de directorios

Primero, cree el directorio donde se almacenar los certificados de Mosquitto:

```
sudo mkdir -p /etc/mosquitto/certs
```

4.2.1. Generación de la Autoridad Certificadora (CA)

1. Crear la clave privada de la CA

Ejecute el siguiente comando para generar una clave RSA de 2048 bits:

```
openssl genrsa -out ca.key 2048
```

2. Crear el certificado de la CA

- Use el siguiente comando para crear el certificado auto firmado de la Autoridad Certificadora. **IMPORTANTE:** cuando se le pida el **Common Name**, ingrese algo como **MiCA_Local**.
- ```
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt
```

Durante la ejecución, se le solicitará información en un formato denominado Distinguished Name (DN). Utilice los siguientes valores de ejemplo:

- **Country Name** (Código de país): CO
- **State or Province Name** (Provincia o estado): Cauca
- **Locality Name** (Localidad): Popayán
- **Organization Name** (Organización): Plc
- **Organizational Unit Name** (Unidad organizacional): Automática
- **Common Name** (Nombre común): Mcp\_IA
- **Email Address** (Correo electrónico): frealpe@unicauca.edu.co

3. Generación del certificado del servidor

Crear la clave privada del servidor, en la misma carpeta de certificados, ejecute:

```
openssl genrsa -out server.key 2048
```

#### 4. Crear la solicitud de firma de certificado (CSR)

- Cree el archivo de solicitud de firma del certificado

```
openssl req -new -out server.csr -key server.key
```

- Ingrese la siguiente información cuando se le solicite

- **Country Name** (Código de país): CO
- **State or Province Name** (Provincia o estado): Cauca
- **Locality Name** (Localidad): Popayán
- **Organization Name** (Organización): Servidor\_MQTT
- **Organizational Unit Name** (Unidad organizacional): Automática
- **Common Name** (Nombre común): 192.168.20.23 (o la dirección IP de su servidor)
- **Email Address** (Correo electrónico): frealpe@unicauca.edu.co

Para los campos de atributos adicionales (**A challenge password** y **An optional company name**), puede dejarlos en blanco presionando Intro.

## 4.3. Configuración de Mosquitto con WebSocket y Seguridad

### 4.3.1. Introducción

Mosquitto es un servidor MQTT ligero y de código abierto que facilita la comunicación entre dispositivos IoT. Para garantizar una comunicación segura y confiable en entornos de producción, es esencial configurar Mosquitto con múltiples puertos que soporten distintos protocolos y niveles de seguridad.

Esta sección describe cómo configurar Mosquitto para soportar cuatro tipos de conexiones:

1. **MQTT estándar (puerto 1883)**: Para comunicación sin cifrado, útil en redes locales confiables.
2. **MQTT Seguro - MQTTS (puerto 8883)**: Para comunicación cifrada mediante TLS, utilizado por dispositivos IoT tradicionales.
3. **WebSocket (puerto 8083)**: Para navegadores web y aplicaciones React sin cifrado.
4. **WebSocket Seguro - WSS (puerto 8084)**: Para navegadores web con cifrado TLS, la opción más segura para aplicaciones web.

Además, implementaremos autenticación basada en usuario y contraseña, así como listas de control de acceso (ACL) para restringir qué clientes pueden publicar o suscribirse a temas específicos.

#### **4.3.2. Archivo de Configuración Principal**

El archivo de configuración de Mosquitto se encuentra en `/etc/mosquitto/mosquitto.conf`. A continuación se detalla la configuración completa con seguridad habilitada:

---

**Algoritmo 4.1** Configuración mosquitto.conf

---

```
=====
Configuración Mosquitto con WebSocket + Seguridad
=====

— Puerto MQTT normal (para Node.js , PLC, etc.)
listener 1883 0.0.0.0
protocol mqtt

— Puerto MQTT SEGURO (MQTTS — El que usa la "llave")
listener 8883 0.0.0.0
protocol mqtt
cafile /etc/mosquitto/certs/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key

— Puerto WebSocket (para navegadores , React , etc.)
listener 8083 0.0.0.0
protocol websockets

— Puerto WebSocket SEGURO (WSS — Para la web con SSL)
listener 8084 0.0.0.0
protocol websockets
cafile /etc/mosquitto/certs/ca.crt
certfile /etc/mosquitto/certs/server.crt
keyfile /etc/mosquitto/certs/server.key

— Seguridad
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/aclfile

— Logs
connection_messages true
log_type all
log_dest syslog
log_dest stdout
```

---

### 4.3.3. Descripción de Directivas de Configuración

#### Listeners (Puertos de Escucha)

Cada `listener` define un puerto sobre el cual Mosquitto escucha conexiones entrantes. La dirección `0.0.0.0` indica que acepta conexiones de cualquier

interfaz de red.

- **listener 1883 0.0.0.0**: Puerto MQTT estándar sin encriptación.
- **listener 8883 0.0.0.0**: Puerto MQTT seguro con soporte TLS.
- **listener 8083 0.0.0.0**: Puerto WebSocket sin encriptación.
- **listener 8084 0.0.0.0**: Puerto WebSocket seguro con TLS.

## Certificados y Claves TLS

Para los puertos seguros (8883 y 8084), se deben especificar tres archivos generados anteriormente:

- **cafile**: Certificado de la Autoridad Certificadora (**ca.crt**).
- **certfile**: Certificado del servidor (**server.crt**).
- **keyfile**: Clave privada del servidor (**server.key**).

Estos archivos deben tener permisos apropiados y ser accesibles solo por el usuario de Mosquitto.

## Autenticación y Control de Acceso

- **allow\_anonymous false**: Desactiva las conexiones anónimas, requiriendo que todos los clientes proporcionen credenciales.
- **password\_file**: Ruta al archivo que contiene los nombres de usuario y contraseñas hasheadas.
- **acl\_file**: Ruta al archivo que define las listas de control de acceso para cada usuario.

## Registro de Eventos (Logging)

- **connection\_messages true**: Registra mensajes de conexión y desconexión de clientes.
- **log\_type all**: Registra todos los tipos de eventos.
- **log\_dest syslog**: Envía los logs al servicio syslog del sistema.
- **log\_dest stdout**: Envía los logs a la salida estándar (útil para debugging).

## Notas importantes

- Asegúrese de que todos los certificados y claves estén almacenados en `/etc/mosquitto/certs`.
- El `Common Name` del servidor debe coincidir con la dirección IP o el nombre de dominio del servidor MQTT.
- Los certificados generados con este procedimiento son válidos por 3650 días (10 años).
- Este proceso establece una Infraestructura de Clave Pública (PKI) para asegurar la comunicación mediante TLS.

## 4.4. Web Socket

### Introducción:

El Pilar de la IoT Moderna La convergencia entre la inteligencia artificial y los sistemas embebidos ha transformado el diseño de circuitos y la gestión de datos en una disciplina que exige tanto potencia de procesamiento como blindaje digital absoluto. En la actualidad, desarrollar un sistema electrónico no solo implica trazar pistas en una PCB o programar la lógica de un microcontrolador, sino también garantizar que cada paquete de información viaje a través de canales cifrados y que los modelos de lenguaje que procesan dicha información operen bajo un esquema de privacidad y baja latencia. La implementación de protocolos seguros y el despliegue de modelos locales representan, por tanto, la diferencia entre un prototipo vulnerable y una solución industrial robusta capaz de operar en entornos críticos sin depender de infraestructuras externas.

Explicación Detallada: Seguridad y Autonomía Para entender cómo se integran estas tecnologías en tus proyectos de desarrollo, debemos desglosar tres pilares fundamentales:

1. Cifrado de Extremo a Extremo (MQTTS y Sockets) Cuando configuras un dispositivo (como un ESP32 o una Raspberry Pi) para comunicarse con un servidor, el uso de TLS/SSL es obligatorio.

El apretón de manos (Handshake): Antes de enviar el primer bit de datos, el cliente y el servidor intercambian certificados. Esto confirma que el servidor es quien dice ser (evitando el phishing de datos).

Túnel de Datos: Una vez establecida la identidad, se genera una llave de sesión única. A partir de ese momento, incluso si alguien intercepta la señal Wi-Fi, solo verá ruido digital ilegible.

2. Inteligencia Artificial Local vs. Cloud La decisión de correr un modelo de IA de forma local (usando herramientas como Ollama o Llama.cpp) impacta directamente en el diseño de tu circuito y la arquitectura de red:

Privacidad: Los datos de tus sensores (como los de NPK o presencia) nunca salen de tu red local.



Determinismo: No dependes de si el servidor de OpenAI está caído o si tu conexión a internet es lenta; la respuesta de la IA es constante y predecible.

Hardware: Requiere que tu "Master" (por ejemplo, una de tus Raspberry Pi o una PC con GPU) tenga suficiente memoria para cargar los pesos del modelo.

3. Validación de Integridad en Código Como vimos en tu fragmento de código, la validación mediante setCACert no es un paso opcional en producción. Al asignar un puerto específico (como el 8883), obligas al hardware a rechazar cualquier conexión que no cumpla con los estándares de seguridad mínimos, creando un ecosistema donde solo los dispositivos autorizados pueden interactuar.

Nota Técnica: Si planeas integrar esto en tu proyecto de Tamagotchi con IA o en el agente de mantenimiento de Factory I/O, el uso de modelos locales te permitirá una interacción mucho más fluida (en milisegundos) en comparación con las llamadas a una API externa.

# Capítulo 5

## Esp32

### 5.1. Qué es PlatformIO? PlatformIO

Es un ecosistema para el desarrollo de sistemas embebidos (como el ESP32) que funciona como una extensión dentro de Visual Studio Code. A diferencia de herramientas básicas, este permite gestionar automáticamente las librerías y las placas, lo cual es ideal para proyectos complejos como el control de procesos industriales o asistentes de IA.

1. Instalación Paso a Paso Descargar VS Code:

Instala Visual Studio Code en tu ordenador.

Instalar la Extensión: Abre VS Code, ve al icono de extensiones (cuadrados en la barra lateral) y busca "PlatformIO IDE". Haz clic en instalar.

Reiniciar: Una vez instalado, aparecerá un pequeño icono de una hormiga en la barra lateral izquierda.

2. Creación de un Nuevo Proyecto Al hacer clic en el icono de la hormiga y seleccionar "Open" > "New Project", deberás completar tres campos clave:

Nombre: Elige un nombre para tu proyecto (ej. Control\_Autonomo\_ESP32).

Placa (Board): Para la mayoría de los módulos comunes, selecciona Espressif ESP32 Dev Module.

Framework: Selecciona Arduino, que es el más amigable para principiantes ver 5.1.

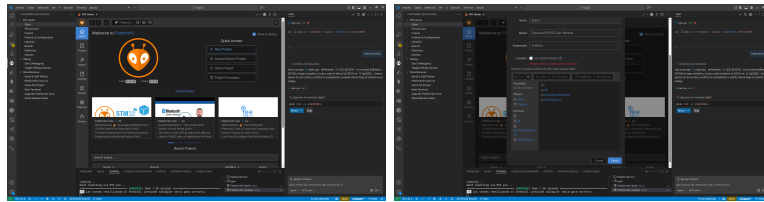


Figura 5.1: Proyecto MCP

3. Estructura del Proyecto Un proyecto de PlatformIO se organiza de forma lógica para evitar el desorden:

src/: Aquí es donde escribes tu código principal (archivo main.cpp).

lib/: Para tus propias librerías locales.

platformio.ini: Este es el archivo de configuración o "cerebro" del proyecto ver 5.2.

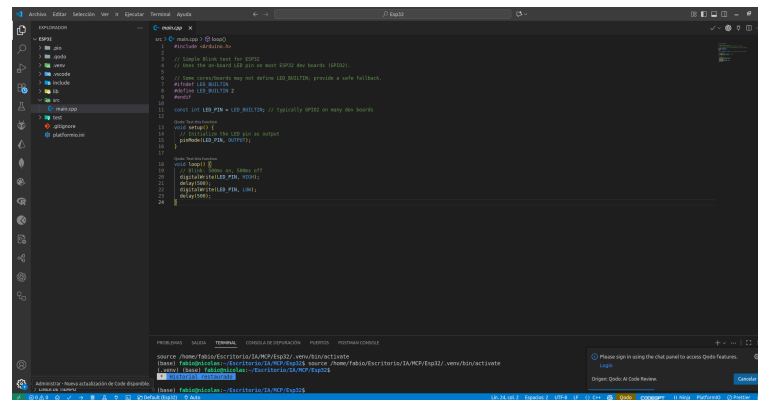


Figura 5.2: Estructura del proyecto

4. El Archivo platformio.ini, es un archivo de texto donde le dices a PlatformIO cómo debe tratar a tu placa. Un ejemplo estándar se ve así 5.1:

---

**Algoritmo 5.1** platformio.ini.

---

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200 ;
upload_port = /dev/ttyUSB0
Configuración para ver datos en consola
```

---

5. Para compilar y cargar el programa por terminal puede escribir las siguientes líneas de código ver.

---

**Algoritmo 5.2** Compilar y cargar

---

```
#1 pio run -e esp32dev
#2 pio run -e esp32dev -t upload
#3 ls -l /dev/ttyUSB* /dev/ttyACM* /dev/serial/by-id || true
#4 pio run -e esp32dev -t upload --upload-port /dev/ttyUSB0
```

---

- a) Compilación del Proyecto `pio run -e esp32dev`
- ¿Qué hace?: Compila (Build) el código fuente del proyecto.
  - Detalle: La opción `-e esp32dev` le indica a PlatformIO que use el entorno específico llamado `esp32dev` definido en tu archivo `platformio.ini`. Sirve para verificar si hay errores de sintaxis sin necesidad de conectar la placa.
- b) Subida del Código (Automática) `pio run -e esp32dev -t upload`
- 1) ¿Qué hace?: Compila el código y luego intenta subirlo (Flash) a la placa conectada.
  - 2) Detalle: El parámetro `-t upload` (target upload) activa el protocolo de transferencia. PlatformIO buscará automáticamente en qué puerto USB está conectada tu placa.
- c) Identificación de Puertos (Solo Linux/macOS) `ls -l /dev/ttyUSB* /dev/ttyACM* /dev/serial/by-id || true`
- 1) ¿Qué hace?: Lista todos los dispositivos seriales y USB conectados al sistema.
  - 2) Detalle: Es una herramienta de diagnóstico. Si tu placa no aparece aquí, es probable que el cable USB esté fallando o falten los drivers (CP2102/CH340). El `|| true` evita que la terminal marque un error si no encuentra ningún dispositivo.
- d) Subida Forzada a un Puerto Específico `pio run -e esp32dev -t upload --upload-port /dev/ttyUSB0`

- 1) ¿Qué hace?: Obliga a PlatformIO a subir el código a un puerto específico, en este caso /dev/ttyUSB0.
- 2) Detalle: Se usa cuando tienes varios dispositivos conectados al mismo tiempo (como dos ESP32 o un sensor NPK USB) y quieres asegurarte de que el código vaya exactamente a la placa de control de tu arquitectura.

6. La configuración del archivo para nuestro proyecto se muestra en

---

### Algoritmo 5.3 Configuración de platformio.in

---

```
;MCP PlatformIO configuration file
; Archivo de configuración de PlatformIO con explicaciones en cada línea
; [platformio] (Ajustes Globales)
; Es la sección de configuración general del proyecto.
; Todo lo que pongas aquí afecta a todo el archivo platformio.ini,
; sin importar qué placa o entorno estés usando.
; Para qué sirve: Define rutas de carpetas (como data_dir)
; o cuál de todos tus entornos debe ser el prioritario por defecto (default_env).
[platformio]
default_env: entorno que se usará por defecto si no se especifica con -e
default_env = esp32dev
data_dir: carpeta donde se almacenan datos del proyecto (ej. archivos para upload)
data_dir = src/data
[common] (Sección de Herencia/Plantilla)
; Esta no es una sección oficial de PlatformIO,
; sino una convención que usamos los desarrolladores para no repetir código.
; Para qué sirve: Es como una "bolsa" donde guardas configuraciones que quieres usar en múltiples placas.
; Por ejemplo, si tienes 5 modelos de ESP32 diferentes, defines la versión o las librerías
; aquí una sola vez y luego las "llamas" desde los otros entornos usando ${common.nombre_de_variable}.
; Dato curioso: Puedes llamarla [common], [config] o como quieras, mientras luego la referencias correctamente.
[common]
version: bandera de preprocesador que se añade a la compilación (ej. para identificar builds)
version = -D BUILD_TAG=V1.0.a0-Build-20260202
lib_deps: dependencias comunes de librerías (vacío si no hay dependencias compartidas)
lib_deps =
; [env:nombre_del_entorno] (Configuración del Entorno)
; Esta es la sección más importante. Cada bloque que empiece con env: define una tarea de compilación específica.
; Para qué sirve: Aquí especificas los "fierros" (el hardware).
; Le dices a PlatformIO: "Para este entorno llamado 'esp32dev',
; usa estas librerías, esta velocidad de subida y este framework".
; Multi-entorno: Puedes tener varios. Por ejemplo: [env:debug] (para pruebas lentas con muchos mensajes).
; [env:release] (para la versión final optimizada).
[env:esp32dev]
platform: plataforma de desarrollo (núcleo y toolchain), aquí Espressif32 para ESP32
platform = espressif32
framework: framework usado por el proyecto (Arduino en este caso)
framework = arduino
board: placa objetivo definida por PlatformIO (define pines, flash, etc.)
board = esp32dev
board_build.mcu: identificador del microcontrolador (opcional, usado para algunos ajustes)
board_build.mcu = esp32
board_build.partitions:
ruta a una tabla de particiones personalizada (vacía = usar la por defecto)
board_build.partitions = default_ota.csv
upload_protocol: protocolo/utilidad que se usa para subir el firmware (esptool es común en ESP32)
upload_protocol = esptool
lib_deps: dependencias de librerías específicas de este entorno; aquí se incluye lo común
lib_deps =
 ${common.lib_deps}
; src_build_flags: flags de compilación/definiciones para el código fuente (usa la variable common.version)
src_build_flags = ${common.version}
upload_speed: baudrate usado por la herramienta de subida (esptool)
upload_speed = 921600
monitor_speed: baudrate para el monitor serie
monitor_speed = 115200
upload_port: puerto serie a usar para subir el firmware.
; Si se deja comentado, PlatformIO intentará autodetectar el puerto.
upload_port = /dev/ttyUSB0
monitor_port: puerto a usar para el monitor serie (comentado para autodetección)
monitor_port = /dev/ttyUSB0
```

---

7. Archivos de distribución de memoria

| Nombre  | Tipo | Subtipo | Offset(inicio) | Tamaño(hex) | Tamaño aprox    | ¿Para qué sirve?                                                    |
|---------|------|---------|----------------|-------------|-----------------|---------------------------------------------------------------------|
| nvs     | data | nvs     | 0x9000         | 0x5000      | 20KB            | Guarda configuraciones persistentes (WiFi, parámetros, calibración) |
| otadata | data | ota     | 0xE000         | 0x2000      | 8KB             | Indica qué firmware OTA está activo                                 |
| ota_0   | app  | ota_0   | 0x10000        | 0x1E0000    | $\approx 1,9MB$ | Firmware principal (aplicación)                                     |
| ota_1   | app  | ota_1   | 0x1F0000       | 0x1E0000    | $\approx 1,9MB$ | Firmware de respaldo / actualización OTA                            |
| spiffs  | data | spiffs  | 0x3D0000       | 0x6000      | 24 KB           | Sistema de archivos (logs, JSON, datos)                             |

Tabla 5.1: Particiones de Esp32

a) El ESP32 tiene memoria flash (por ejemplo 4 MB). Esta tabla le dice al sistema:

- dónde guardar configuraciones
- dónde va el programa
- dónde va el firmware OTA
- dónde guardar archivos (SPIFFS)

b) NVS = Non-Volatile Storage

- Guarda configuraciones persistentes
- Variables que no se borran al apagar
- Ej: WiFi, credenciales, calibraciones, parámetros
- Preferences
- WiFi.begin()

c) otadata

- Aquí el ESP32 guarda cuál firmware OTA está activo
- Indica si está corriendo ota0 u ota1
- Sin esto, OTA no funciona
  - ota0:
    - Primer espacio para tu programa principal
    - Aquí se graba el firmware
    - Tamaño  $\approx 1,9MB$
    - Es el firmware activo o uno de respaldo
  - ota1
    - Permite actualizar sin borrar el anterior
    - Si falla la actualización vuelve al anterior
    - Esto es clave para sistemas industriales / IIoT

d) spiffs:Sistema de archivos SPIFFS

- Guarda archivos
- JSON
- HTML
- CSS
- logs
- modelos pequeños (ej. coeficientes)
- Se mantiene tras reinicios

8. Según la estructura del proyecto(control, IA, IIoT)

- OTA te permite actualizar controladores sin parar planta
- NVS es ideal para:
  - parámetros de control
  - ganancias PI
  - coeficientes AR/IA
- SPIFFS sirve para:
  - logs
  - perfiles
  - datos históricos

9. Versionamiento de firmware ver 5.4

---

**Algoritmo 5.4** Versionamiento

---

```
;Determinar Versión de Firmware
; X.Y.Z
; (X) versión mayor
; (Y) versión menor
; (Z) revisión
; Alpha, Beta,
;RC (Alpha es una versión inestable
;Beta una versión mas estable que Alpha
; RC (Release Candidate))
; v1.0.Alpha – v1.0.a0
; v2.1.Beta – v1.0.b1
; v3.0–RC – v3.0.rc3
; Fecha: Año–mes–día ; v1.0.0a–20260202
```

---

## 5.2. Esquema del Proyecto

La figura 5.3muestra de manera general la arquitectura del proyecto para el uso de la esp32 en modo Ap o Cliente.

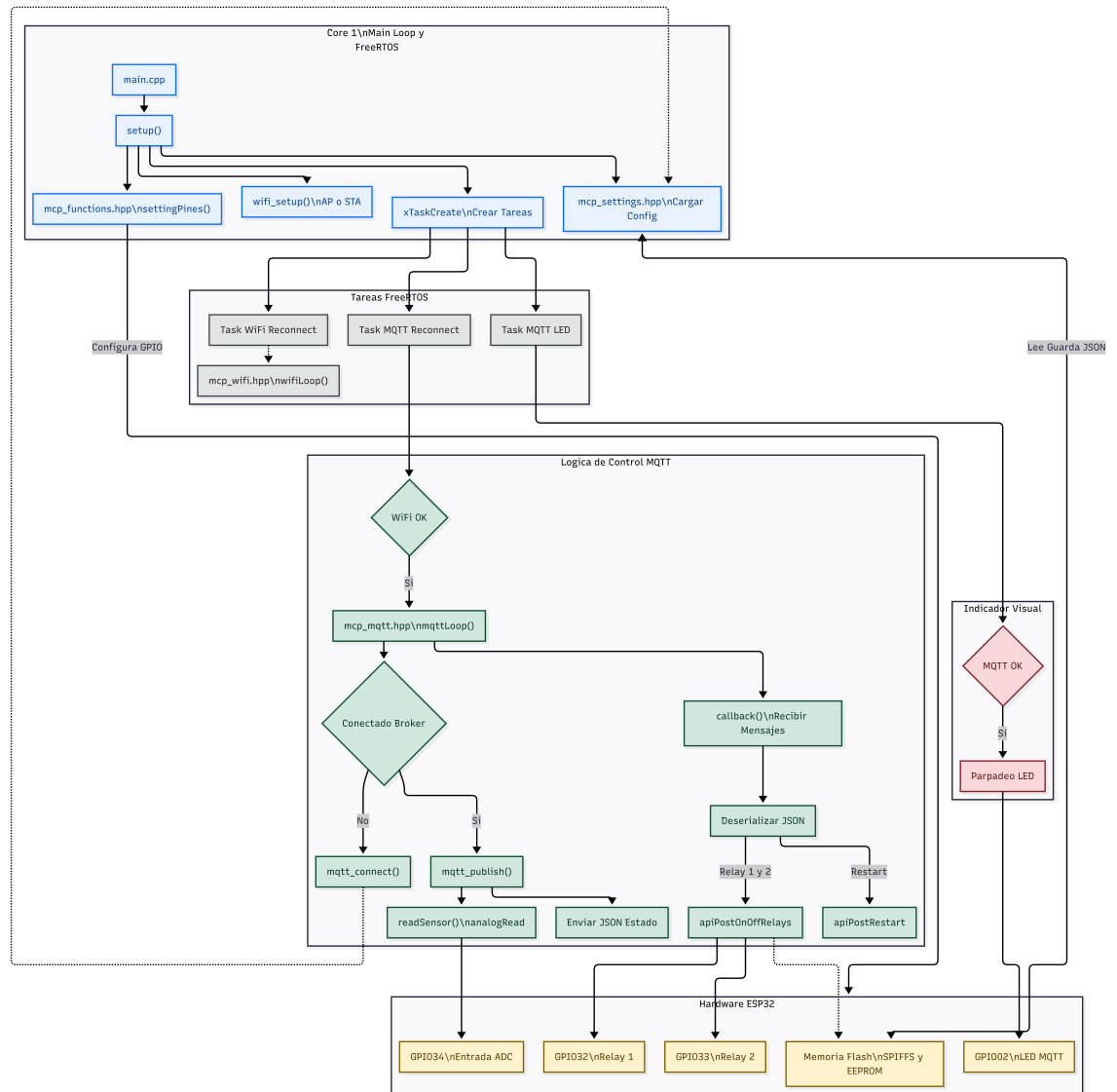


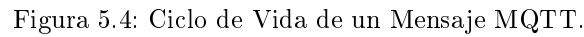
Figura 5.3: Diagrama principal del proyecto



1. La Jerarquía de Red y el Broker MQTT En una arquitectura MQTT, el Broker es el nodo central (Hub). Para que un ESP32 funcione como cliente, debe existir una infraestructura de red subyacente que permita el transporte de capas superiores (TCP/IP).
  - En Modo STA: El ESP32 delega la gestión de la red al Router (Gateway). El Router se encarga de la asignación de IPs vía DHCP, la resolución de nombres (DNS) y, lo más importante, el enrutamiento hacia el exterior. Esto permite que el ESP32 alcance brokers en la nube (como AWS IoT, HiveMQ o Adafruit IO) o en servidores locales (como Raspberry Pi con Mosquitto).
  - En Modo AP: El ESP32 asume el rol de Gateway. Sin embargo, por defecto, un ESP32 en modo AP no tiene una "salida" a internet a menos que actúe como puente (Bridge), lo cual consume recursos críticos de la CPU y memoria RAM, limitando la estabilidad de la conexión MQTT.
2. El Ciclo de Vida de la Conexión (Stack TCP/IP) Técnicamente, el proceso de reconexión que mencionas (TaskMqttReconnect) opera sobre varias capas del modelo OSI:
  - Capa 2 (Enlace): El ESP32 negocia la autenticación WPA2 con el Router.
  - Capa 3 (Red): El stack LwIP (Lightweight IP) del ESP32 obtiene una dirección IP. Sin una IP válida en la subred del Broker, el socket TCP no puede abrirse.
  - Capa 4 (Transporte): Se establece un Handshake TCP en el puerto 1883 (o 8883 para TLS).
  - Capa 7 (Aplicación): Se envía el paquete CONNECT de MQTT.

Si el ESP32 estuviera en modo AP, el Broker tendría que "saltar" a la red generada por el ESP32, lo que rompería la escalabilidad: un Broker suele manejar cientos de clientes; no puede estar saltando de red en red.
3. Gestión de Recursos y Eficiencia El uso de MQTT en modo STA permite aprovechar funciones avanzadas de ahorro de energía:
  - Modem-Sleep: En modo STA, el ESP32 puede entrar en un estado de bajo consumo entre los intervalos de "Keep Alive" de MQTT, manteniendo la asociación con el Router. En modo AP, la radio debe estar activa constantemente para mantener la baliza (Beacon) de la red visible, lo que drena la batería rápidamente.
  - Determinismo en la Tarea: Tu TaskMqttReconnect actúa como una Máquina de Estados. Verifica el bit de evento de WiFi (WIFI\_CONNECTED\_BIT);

4. El flujo de información bidireccional se muestra en 5.4



### 5.3.1. Configuración de variables globales para las conexión wi-fi

A continuación se describe `mcp_header.hpp`, la cual es una librería de parámetros globales que permite la configuración de las conexiones de protocolo de comunicación ver 5.5

---

## Algoritmo 5.5 Configuraciones globales del dispositivo

---

```
// -----
// Definiciones
// -----
#define WIFILED 12 // GPIO12 LED WIFI
#define MQTTLED 13 // GPIO13 LED MQTT
// -----
// CALCULAR LA CAPACIDAD DEL JSON
// Asistente ArduinoJson: https://arduinojson.org/v6/assistant/
// Documentación: https://arduinojson.org/v6/api/json/deserializejson/
// -----
const size_t capacitySettings = 1536;
// -----
// Versión de Firmware desde las variables de entorno platformio.ini
// -----
#define TEXTIFY(A) #A
#define ESCAPEQUOTE(A)
TEXTIFY(A)
String device_fw_version = ESCAPEQUOTE(BUILD_TAG);
// -----
// Version de Hardware y Fabricante
// -----
#define device_hw_version "MCP32 v1 00000000" // Versión del hardware
#define device_manufacturer "IOTTSolutions" // Fabricante
// -----
// Zona configuración Dispositivo
// -----
boolean device_config_file; // Identificador para archivo de configuración
char device_config_serial[30]; // Numero de serie de cada Archivo configuración unico
char device_id[30]; // ID del dispositivo
int device_restart; // Número de reinicios
char device_old_user[15]; // Usuario para acceso al servidor Web char
char device_old_password[15]; // Contraseña del usuario servidor Web
// -----
// Zona configuración WIFI modo Cliente
// -----
boolean wifi_ip_static; // Uso de IP Estática DHCP
char wifi_ssid[30]; // Nombre de la red WiFi
char wifi_password[30]; // Contraseña de la Red WiFi
char wifi_ipv4[15]; // Dir IPv4 Estático
char wifi_gateway[15]; // Dir IPv4 Gateway
char wifi_subnet[15]; // Dir IPv4 Subred
char wifi_dns_primary[15]; // Dir IPv4 DNS primario
char wifi_dns_secondary[15]; // Dir IPv4 DNS secundario
// -----
// Zona configuración WIFI modo AP
// -----
boolean ap_mode; // Uso de Modo AP
char ap_ssid[31]; // Nombre del SSID AP
char ap_password[63]; // Contraseña del AP
int ap_channel; // Canal AP
int ap_visibility; // Es visible o no el AP (0 - Visible 1 - Oculto)
int ap_connect; // Número de conexiones en el AP máx 8 conexiones ESP32
// -----
// Zona configuración MQTT
// -----
boolean mqtt_cloud_enable; // Habilitar MQTT Broker
char mqtt_cloud_id[30]; // Cliente ID MQTT Broker
char mqtt_user[30]; // Usuario MQTT Broker
char mqtt_password[39]; // Contraseña del MQTT Broker
char mqtt_server[39]; // Servidor del MQTT Broker
int mqtt_port; // Puerto servidor MQTT Broker
boolean mqtt_retain; // Habilitar mensajes retenidos
int mqtt_qos; // Calidad del servicio
boolean mqtt_time_send; // Habilitar en envio de datos
int mqtt_time_interval; // Tiempo de envio por MQTT en segundos
boolean mqtt_status_send; // Habilitar en envio de estados
// -----
// Zona Firmware Update TODO: Para La API REST
// -----
size_t content_len;
#define U_PART U_SPIFFS // Unidad de almacenamiento para Firmware
// Update U_PART formatea en firmware U_SPIFFS formatea en el SPIFFS
// -----
// Zona EEPROM para contador de reinicios
// -----
#define Start_Address 0
#define Restart_Address Start_Address + sizeof(int)
```

---

### 5.3.2. Configuración de Lectura, Escritura y Reset de parámetros.

La librería ArduinoJson permite de manera eficiente en un entorno industrial o de desarrollo con microcontroladores (como un Raspberry Pi PLC o ESP32), configurar de manera inteligente la gestión de la memoria <https://arduinojson.org/v7/assistant/#/step1>.

1. Configuración de Memoria: Static vs Dynamic: La elección del "Document" es el paso más crítico en ArduinoJson 7.

- JsonDocument (Recomendado para V7): En la versión más reciente, ya no necesitas preocuparte por elegir entre Static o Dynamic en la mayoría de los casos; el objeto JsonDocument gestiona el heap de forma automática y eficiente.
- Capacidad: Aunque es dinámico, si conoces el tamaño máximo de tus archivos, es una buena práctica pre-asignar espacio para evitar fragmentación.
- Se debe agregar la primera librería que permita trabajar con archivos Json, se debe configurar el archivo platformio.ini.

- `platformio.ini`  
lib\_deps =  
bblanchon/ArduinoJson@^6.21.5

Para la configuración de los parámetros de conexión wifi mediante archivos Json tendremos la siguiente arquitectura.

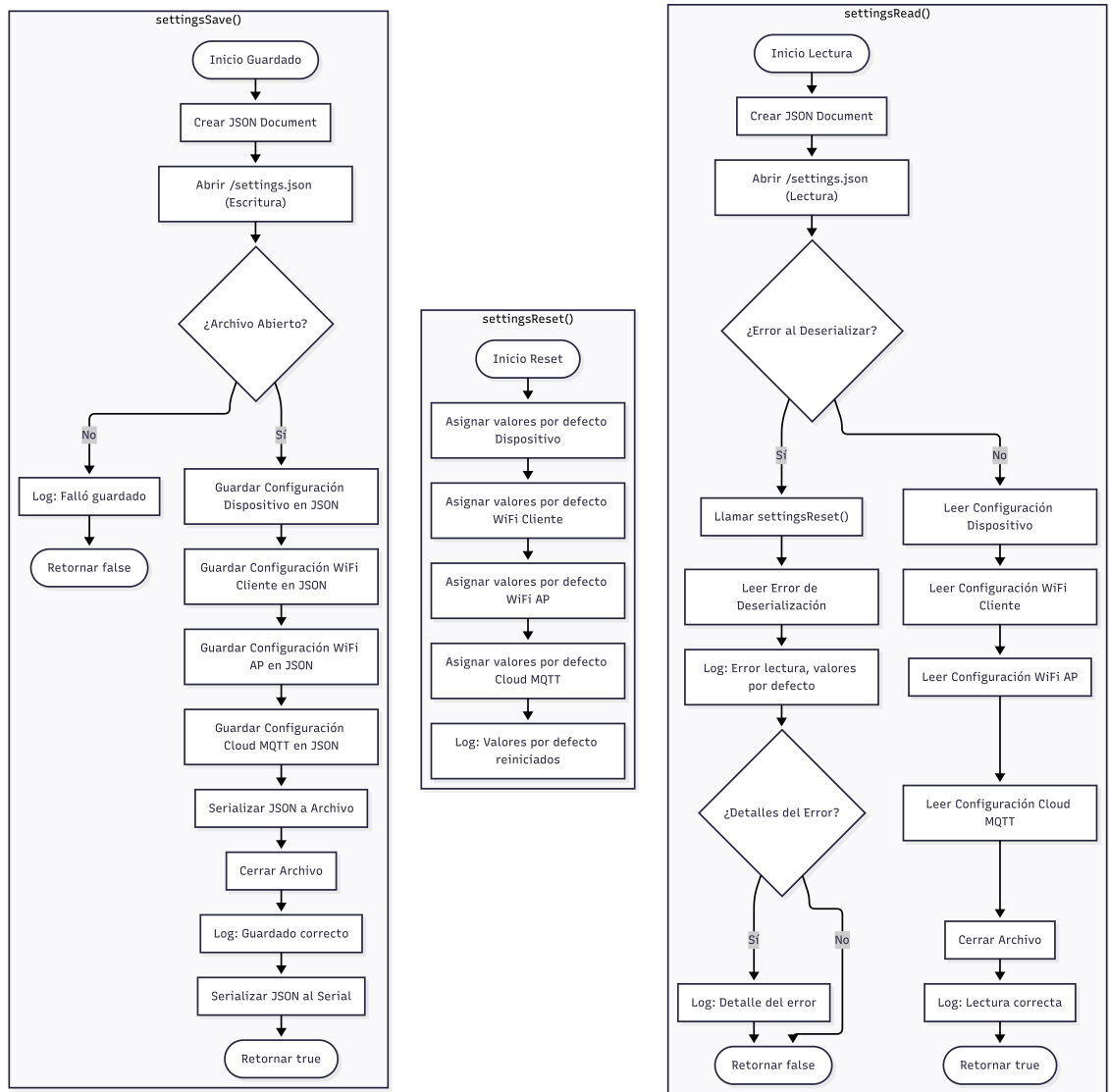


Figura 5.5: Arquitectura de configuración Wiffi

La figura 5.5 muestra las tres funciones asociadas a la configuración del esp32 de la librería mcp\_settings.hpp ver 5.6, se muestra las formas de conexión a la red ya sea como usuario o punto de acceso. De la misma manera se establece el protocolo Mqtt Si no llega ninguna configuración toma los parámetros por defecto de fabrica

El código asociado a la función boolean settingsRead se muestra en 5.7.

---

## Algoritmo 5.6 mcp\_settings.hpp

---

```
boolean settingsRead();
//Leer parámetros
void settingsReset();
//Reiniciar parámetros
boolean settingsSave();
//Guardar parámetros
```

---

---

## Algoritmo 5.7 settingsRead

---

```
boolean settingsRead(){
 DynamicJsonDocument jsonSettings(capacitySettings);
 File file = SPIFFS.open("/settings.json", "r");

 if(deserializeJson(jsonSettings, file)){
 // Tomar los valores de Fábrica
 settingsReset();
 DeserializationError error = deserializeJson(jsonSettings, file);
 log("[ERROR] Falló la lectura de las configuraciones, tomando
 valores por defecto");
 if(error){
 log("[ERROR] " + String(error.c_str()));
 }
 return false;
 }else{
 //=====
 // Dispositivo settings.json
 //=====
 device_config_file = jsonSettings["device_config_file"];
 strcpy(device_config_serial, jsonSettings["device_config_serial"], sizeof(
 device_config_serial));
 strcpy(device_id, jsonSettings["device_id"], sizeof(device_id));
 strcpy(device_old_user, jsonSettings["device_old_user"], sizeof(device_old_user));
 strcpy(device_old_password, jsonSettings["device_old_password"], sizeof(
 device_old_password));
 //=====
 // WIFI Cliente settings.json
 //=====
 wifi_ip_static = jsonSettings["wifi_ip_static"];
 strcpy(wifi_ssid, jsonSettings["wifi_ssid"], sizeof(wifi_ssid));
 strcpy(wifi_password, jsonSettings["wifi_password"], sizeof(wifi_password));
 strcpy(wifi_ipv4, jsonSettings["wifi_ipv4"], sizeof(wifi_ipv4));
 strcpy(wifi_subnet, jsonSettings["wifi_subnet"], sizeof(wifi_subnet));
 strcpy(wifi_gateway, jsonSettings["wifi_gateway"], sizeof(wifi_gateway));
 strcpy(wifi_dns_primary, jsonSettings["wifi_dns_primary"], sizeof(wifi_dns_primary)
);
 strcpy(wifi_dns_secondary, jsonSettings["wifi_dns_secondary"], sizeof(
 wifi_dns_secondary));
 //=====
 // WIFI AP settings.json
 //=====
 ap_mode = jsonSettings["ap_mode"];
 strcpy(ap_ssid, jsonSettings["ap_ssid"], sizeof(ap_ssid));
 strcpy(ap_password, jsonSettings["ap_password"], sizeof(ap_password));
 ap_visibility = jsonSettings["ap_visibility"];
 ap_chanel = jsonSettings["ap_chanel"];
 ap_connect = jsonSettings["ap_connect"];
 //=====
 // Cloud settings.json
 //=====
 mqtt_cloud_enable = jsonSettings["mqtt_cloud_enable"];
 strcpy(mqtt_user, jsonSettings["mqtt_user"], sizeof(mqtt_user));
 strcpy(mqtt_password, jsonSettings["mqtt_password"], sizeof(mqtt_password));
 strcpy(mqtt_server, jsonSettings["mqtt_server"], sizeof(mqtt_server));
 strcpy(mqtt_cloud_id, jsonSettings["mqtt_cloud_id"], sizeof(mqtt_cloud_id));
 mqtt_port = jsonSettings["mqtt_port"];
 mqtt_retain = jsonSettings["mqtt_retain"];
 mqtt_qos = jsonSettings["mqtt_qos"];
 mqtt_time_send = jsonSettings["mqtt_time_send"];
 mqtt_time_interval = jsonSettings["mqtt_time_interval"];
 mqtt_status_send = jsonSettings["mqtt_status_send"];
 file.close();
 log("[INFO] Lectura de las configuraciones correcta");
 return true;
 }
}
```

---

El código asociado a la función void settingsReset se muestra en 5.8

---

**Algoritmo 5.8** settingsReset

---

```
void settingsReset() ;

void settingsReset() {
// -----
// Dispositivo settings.json
// -----
device_config_file = true;
// Habilita el archivo de configuración
strcpy(device_config_serial, deviceID().c_str(),
sizeof(device_config_serial));
strcpy(device_id, "adminmcp", sizeof(device_id));
strcpy(device_old_user, "admin", sizeof(device_old_user));
strcpy(device_old_password, "admin", sizeof(device_old_password));
// -----
// WIFI Cliente settings.json
// -----
wifi_ip_static = jsonSettings["wifi_ip_static"];
strcpy(wifi_ssid, jsonSettings["wifi_ssid"],
sizeof(wifi_ssid));
strcpy(wifi_password, jsonSettings["wifi_password"],
sizeof(wifi_password));
strcpy(wifi_ipv4, jsonSettings["wifi_ipv4"],
sizeof(wifi_ipv4));
strcpy(wifi_subnet, jsonSettings["wifi_subnet"],
sizeof(wifi_subnet));
strcpy(wifi_gateway, jsonSettings["wifi_gateway"],
sizeof(wifi_gateway));
strcpy(wifi_dns_primary, jsonSettings["wifi_dns_primary"],
sizeof(wifi_dns_primary));
strcpy(wifi_dns_secondary, jsonSettings["wifi_dns_secondary"],
sizeof(wifi_dns_secondary));
// -----
// WIFI AP settings.json
// -----
ap_mode = jsonSettings["ap_mode"];
strcpy(ap_ssid, jsonSettings["ap_ssid"], sizeof(ap_ssid));
strcpy(ap_password, jsonSettings["ap_password"],
sizeof(ap_password));
ap_visibility = jsonSettings["ap_visibility"];
ap_chanel = jsonSettings["ap_chanel"];
ap_connect = jsonSettings["ap_connect"];
// -----
// Cloud settings.json
// -----
mqtt_cloud_enable = jsonSettings["mqtt_cloud_enable"];
strcpy(mqtt_user, jsonSettings["mqtt_user"],
sizeof(mqtt_user));
strcpy(mqtt_password, jsonSettings["mqtt_password"],
sizeof(mqtt_password));
strcpy(mqtt_server, jsonSettings["mqtt_server"],
sizeof(mqtt_server));
strcpy(mqtt_cloud_id, jsonSettings["mqtt_cloud_id"],
sizeof(mqtt_cloud_id));
mqtt_port = jsonSettings["mqtt_port"];
mqtt_retain = jsonSettings["mqtt_retain"];
mqtt_qos = jsonSettings["mqtt_qos"];
mqtt_time_send = jsonSettings["mqtt_time_send"];
mqtt_time_interval = jsonSettings["mqtt_time_interval"];
mqtt_status_send = jsonSettings["mqtt_status_send"];
file.close();
log("[INFO] Lectura de las configuraciones correcta");
return true;
}
}
```

---

El código asociado a la función void settingsSave se muestra en 5.9

---

### Algoritmo 5.9 settingsSave

---

```
// Reiniciar parámetros
boolean settingsSave() {
 // StaticJsonDocument<capacitySettings> jsonSettings;
 DynamicJsonDocument jsonSettings(capacitySettings);
 File file =
 SPIFFS.open("/settings.json", "w+");
 if (file) { // Si el archivo se abrió correctamente
 // -----
 // Dispositivo settings.json
 // -----
 jsonSettings["device_config_file"] =
 device_config_file; // Guardar config file
 jsonSettings["device_config_serial"] =
 device_config_serial;
 jsonSettings["device_id"] = device_id;
 jsonSettings["device_old_user"] = device_old_user;
 jsonSettings["device_old_password"] =
 device_old_password; // Guardar old password
 // -----
 // WIFI Cliente settings.json
 // -----
 jsonSettings["wifi_ip_static"] = wifi_ip_static;
 jsonSettings["wifi_ssid"] = wifi_ssid;
 jsonSettings["wifi_password"] = wifi_password;
 jsonSettings["wifi_ipv4"] = wifi_ipv4;
 jsonSettings["wifi_subnet"] = wifi_subnet;
 jsonSettings["wifi_gateway"] = wifi_gateway;
 jsonSettings["wifi_dns_primary"] = wifi_dns_primary;
 jsonSettings["wifi_dns_secondary"] =
 wifi_dns_secondary; // Guardar DNS secundario
 // -----
 // WIFI AP settings.json
 // -----
 jsonSettings["ap_mode"] = ap_mode;
 jsonSettings["ap_ssid"] = ap_ssid;
 jsonSettings["ap_password"] = ap_password;
 jsonSettings["ap_visibility"] = ap_visibility;
 jsonSettings["ap_chanel"] = ap_chanel;
 jsonSettings["ap_connect"] = ap_connect;
 // -----
 // Cloud settings.json
 // -----
 jsonSettings["mqtt_cloud_enable"] =
 mqtt_cloud_enable;
 jsonSettings["mqtt_user"] = mqtt_user;
 jsonSettings["mqtt_password"] = mqtt_password;
 jsonSettings["mqtt_server"] = mqtt_server;
 jsonSettings["mqtt_cloud_id"] = mqtt_cloud_id;
 jsonSettings["mqtt_port"] = mqtt_port;
 jsonSettings["mqtt_retain"] = mqtt_retain;
 jsonSettings["mqtt_qos"] = mqtt_qos;
 jsonSettings["mqtt_time_send"] = mqtt_time_send;
 jsonSettings["mqtt_time_interval"] =
 mqtt_time_interval;
 jsonSettings["mqtt_status_send"] = mqtt_status_send;
 serializeJsonPretty(jsonSettings, file);
 file.close();
 log("[INFO] Configuración Guardada correctamente");
 serializeJsonPretty(jsonSettings, Serial);
 return true;
 } else {
 log("[ERROR] Falló el guardado de la configuración");
 return false;
 }
}
```

---

### 5.3.3. Ciclo de Vida del MQTT

La grafica 5.6 muestra el ciclo de vida del mensaje tanto de la transmisión y recepción de los mensajes mediante el protocolo de comunicación MQTT



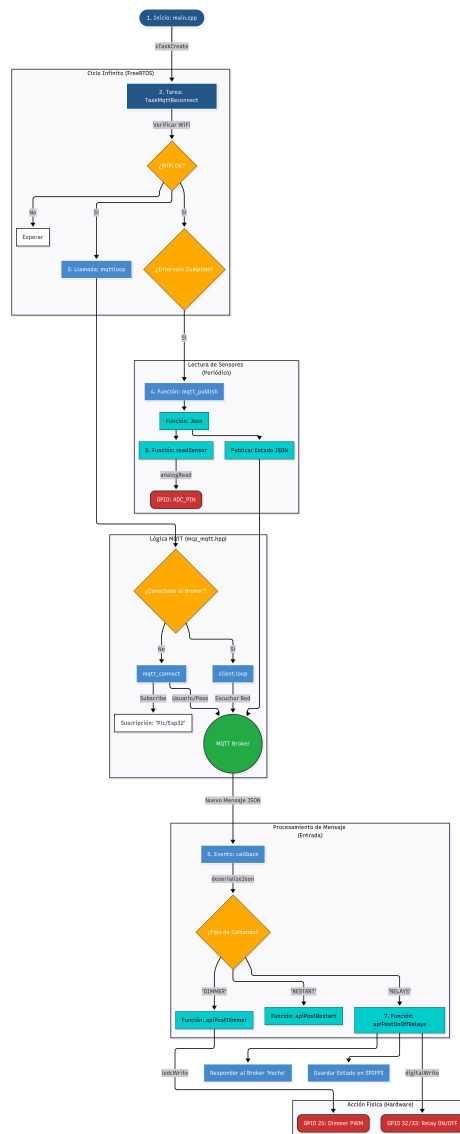


Figura 5.6: Ciclo de vida de Mqtt

## Ciclo infinito (FreeRtos)

1. Empieza con una función de FreeRTOS para crear una "Tarea" (un hilo de ejecución), `xTaskCreate(TaskMqttReconnect, "TaskMqttReconnect", 1024 * 6, NULL, 2, NULL)`.

1 TaskMqttReconnect :

- ¿Qué es?: La función que contiene el código de la tarea.
- En resumen: "Oye ESP32, quiero que ejecutes ESTA función en paralelo".

## 2 "TaskMqttReconnect":

- ¿Qué es?: Un nombre en texto simple para la tarea.
- Para qué sirve: Solo para depuración (debug). Si el sistema falla, te dirá "Falló en la tarea llamada 'TaskMqttReconnect'".

## 3 1024 \* 6 (6144):

- ¿Qué es?: El tamaño de la Memoria Stack (pila) asignada a esta tarea en Bytes.
- Importante: Como tu tarea usa JSON (DynamicJsonDocument), strings y WiFi, necesita bastante memoria. 6KB (6144) es un valor seguro para evitar que se desborde (Stack Overflow) y el ESP32 se reinicie.

## 4 NULL:

- ¿Qué es?: Parámetros que le podrías pasar a la función.
- En este caso: No necesitas pasarle ninguna variable extra al iniciar, así que se deja vacío (NULL).

## 5 2:

- ¿Qué es?: La Prioridad de la tarea.
- Escala: Números más altos = Mayor urgencia.
  - La tarea WiFi tiene prioridad 3 (Más importante).
  - Esta tarea MQTT tiene prioridad 2.
  - La tarea del LED tiene prioridad 1 (Menos importante).
  - Significado: Si la CPU está muy ocupada, atenderá primero al WiFi, luego al MQTT y por último al LED.

## 6 NULL:

- ¿Qué es?: El "Manejador" (Handle) de la tarea.
- Para qué sirve: Si quisieras pausar, reanudar o eliminar esta tarea desde otra parte del código, necesitarías guardar su referencia aquí. Como no planeas manipularla externamente, se deja en NULL.

---

**Algoritmo 5.10** Función TaskMqttReconnect.

---

```
// mcp_tasks.hpp
void TaskMqttReconnect(void *pvParameters) {
 (void)pvParameters;
 while (1) {
 if ((WiFi.status() == WL_CONNECTED)) {
 if (mqtt_server != 0) {
 mqttloop();
 }
 if (mqttClient.connected()) {
 if (millis() - lasMsg > mqtt_time_interval) {
 lasMsg = millis();
 mqtt_publish();
 log("INFO: Mensaje enviado por MQTT...");
 }
 }
 }
 }
}
```

---

Esta tarea de FreeRTOS actúa como un monitor persistente que gestiona la comunicación con el exterior. Primero, valida la Capa de Enlace verificando que el WiFi esté conectado; si es así, y existe una dirección de servidor válida, ejecuta `mqttloop()` para procesar los paquetes de red pendientes y mantener latente la sesión con el Broker. Una vez confirmada la conexión activa en la Capa de Aplicación, utiliza un mecanismo de conteo de tiempo no bloqueante (`millis()`) para enviar datos a través de `mqtt_publish()` solo cuando se cumple el intervalo definido, evitando saturar la red y permitiendo que el sistema siga respondiendo a otros procesos sin detenerse.

2. La función 5.10 llama internamente a `mqttloop`.

1 `mqttloop` la función 5.11 hace parte de la librería de configuración `mcp_mqtt.hpp`

---

**Algoritmo 5.11** Función `mqttloop()`.

---

```
void mqttloop() {
 if (mqtt_cloud_enable) {
 if (!mqttClient.connected()) {
 long now = millis();
 if ((now < 60000) || ((now -
 lastMqttReconnectAttempt) > 120000)) {
 lastMqttReconnectAttempt = now;
 if (mqtt_connect()) {
 lastMqttReconnectAttempt = 0;
 }
 // setOnSingle(MQTTLED);
 }
 } else {
 mqttClient.loop();
 // setOffSingle(MQTTLED);
 }
 }
}
```

---

Esta función actúa como el motor de gestión de conectividad del cliente MQTT, operando bajo una lógica de reconexión inteligente y no bloqueante. Su flujo principal valida primero si el servicio está habilitado y verifica el estado de la sesión; si la conexión se ha perdido, emplea un algoritmo de temporización basado en `millis()` que limita los intentos de reconexión (una vez cada 120 minutos tras el primer minuto de arranque), evitando así que el procesador sature el stack de red con peticiones fallidas constantes. Cuando la sesión es exitosa, la función delega el control a `mqttClient.loop()`, que es el método encargado de mantener vivo el "Keep Alive" con el servidor, procesar los mensajes entrantes y gestionar los buffers de salida, asegurando una comunicación fluida y estable con el broker.

2 Se llama entonces a la función `mqtt_connect{()}` la cual se describe en 5.12 4.2

---

**Algoritmo 5.12** Función `mqtt_connect()`.

---

```
const char *root_ca = "-----BEGIN CERTIFICATE-----\n" "MIID+\n"
TCCAuGgAwIBAgIUC8dGw8Wq8dHaQpaeZ+ZAriKw+/QwDQYJKoZIhvcNAQEL\n"
"BQAwgYsxCzAJBgNVBAYTAkNPMQ4wDAYDVQQIDAVDYXVjYTEQMA4GA1UEBwwHUG9w\n"
"YXlhbjEMMAoGA1UECgwDUGxjMRMwEQYDVQQLDAPBdXRvbWV0aWNhMQ8wDQYDVQQD\n"
"DAZNY3BfSUEXJjAkBgkqhkiG9w0BCQEWf2ZyZWVscGVAdW5pY2F1Y2EuZWR1LmNv\n"
"MB4XDTE2MDIxNDAwNDI1MFoXDTM2MDIxMjAwNDI1MFowgYsxCzAJBgNVBAYTAkNP\n"
"MQ4wDAYDVQQIDAVDYXVjYTEQMA4GA1UEBwwHUG9wYXlhbjEMMAoGA1UECgwDUGxj\n"
"MRMwEQYDVQQLDAPBdXRvbWV0aWNhMQ8wDQYDVQQDDAZNY3BfSUEXJjAkBgkqhkiG\n"
"9w0BCQEWf2ZyZWVscGVAdW5pY2F1Y2EuZWR1LmNvMIIBIjANBgkqhkiG9w0BAQE\n"
"FAAOCAQ8AMIIBCgKCAQEAuaJtcSkurCxuOuZ4mV6xhSkW3fAH9uF2hU8wKy201mGq\n"
"cwLdY2dk6K/MhXS+/0\n"
RfTxRG02sqzDDVnMv350wYo6EecroygonRmOVXSTlafKyC\n" "0p7GqQGOL/\n"
XsuhhDorJvnEuqhIQGFKUSrZO6lvzEmxguCt/vJj5zsoRMaT3izjuP\n"
se0uku0xsAJVp37UDIYdaGACz2y6N94x7aFtaKY38iMvZJx3+jejc6r9/ZVRP\n"
"2r\n"
"DV4SjfrnFejy4uK9ZLDTsIgXTpZla6pz38Wgfv9zifMltqipVOgurLX+4IV\n"
"+5rH\n"
"bIJAg6qG4VOyw76wK0puc+\n"
mfHhRwTRr9k54zmYsnwIDAQABo1MwUTAdBgNVHQ4E\n"
FgQUcMRAgf4Hsr7NMzeBgROxQfygfm4wHwYDVR0jBBgwFoAUcMRAgf4Hsr7NMzeB\n"
"rGOxQfygfm4wDwYDVR0TAQH/BAUwAwEB/\n"
zANBgkqhkiG9w0BAQsFAAOCAQEATE7w\n"
FwUGRrWaucsGJMgjY9f3TuqWwcCW1dZ58MkXawJjNFQ6nXnKsfdNC4nw046dKRax\n"
"z3khg2bUfdArnaAkPx4taFFvP7OSyFIFlbN7zmvKkuDgZ2JsHXNRzjLGEW3gj2SR\n"
"014AO3E2doCx0gVO0vEbs6kJC/u91Jj2QRdfz+\n"
AbQeQn7FDu0Fr3MG8nN4gRuZk\n"
"vv7xZLM/VwllPKC/\n"
DkNXMAzC3hYdvO3eQQRde01oDH9nVmhl/2XSbbKN8/UsON\n"
"8w6h+\n"
PU84b1zacd/LiCwTdDB3+3V0xCzbEUJouMf44pyYyBFVIMX+2LvovNSz+1\n"
"50cZWe7HxAWGJaGUg==\n" "-----END CERTIFICATE-----\n";
```

```
boolean mqtt_connect() {
 if (mqtt_port == 8883) {
 wifiClient.setCACert(root_ca);
 log("MQTT: Usando conexión segura (TLS)");
 } else {
 wifiClient.setInsecure(); // Opcional: permitir conexiones sin
 // no es puerto 8883 // log("MQTT: Usando
 // conexión normal");
 }
 mqttClient.setServer(mqtt_server, mqtt_port);
 mqttClient.setCallback(callback);
 mqttClient.setBufferSize(1024 * 5); // Establecer buffer antes de
 // conectar
 log("MQTT: Intentando conexión al Broker MQTT...");
 if (mqttClient.connect(mqtt_cloud_id, mqtt_user, mqtt_password,
 mqtt_willTopic.c_str(), mqtt_willQoS, mqtt_willRetain,
 mqtt_willMessage.c_str())) {
 log("INFO: Conectado al Broker MQTT -> " + String(mqtt_server));
 }
 mqttClient.setBufferSize(1024 * 5);
 log("INFO: Buffer MQTT Size: " + String(mqttClient.getBufferSize()) +
 " Bytes");
 String topic_subscribe = String(mqtt_topic);
 topic_subscribe.toCharArray(topic, 150);
 if (mqttClient.subscribe(topic)) {
 log("INFO: Suscrito al tópico: " + String(topic));
 } else {
 log("ERROR: MQTT - Falló la suscripción");
 }
 String mqtt_willMessageCon =
 "{\n \"connected\": true,\n \"username\": \"" + String(
 mqtt_user) + "\"\n}";
 mqttClient.publish(mqtt_willTopic.c_str(), mqtt_willMessageCon.c_str());
 } else {
 log("ERROR: MQTT - Falló, código de error = " + String(
 mqttClient.state()));
 return (0);
 }
}
return (1);
}
```

El fragmento de código implementa una lógica de conmutación de seguridad en la capa de transporte del cliente MQTT. El flujo condiciona la configuración del cliente Wi-Fi basándose en el puerto de red definido: si se detecta el puerto 8883 (estándar para tráfico cifrado), se carga un certificado raíz (`root_ca`) que permite al dispositivo verificar la identidad del broker y establecer un túnel TLS/SSL, garantizando la autenticidad y el cifrado de los datos. Por el contrario, si se utiliza cualquier otro puerto, el código desactiva las validaciones de seguridad mediante `setInsecure()`, permitiendo una conexión rápida y sin cifrado, lo cual es útil para pruebas en redes locales controladas pero deja la comunicación vulnerable a interceptaciones en entornos de producción.

Componentes clave del código: `setCACert(root_ca)`: Es el "pasaporte" que el dispositivo usa para confiar en el servidor. Sin esto, el ESP32 no sabría si el broker es el original o un impostor.

`setInsecure()`: Indica al stack de red que ignore la verificación de certificados. Es cómodo para desarrollo, pero arriesgado porque no hay cifrado real de punto a punto.

Puerto 8883: Es la convención de IANA para MQTTS. Usar el puerto 1883 (el estándar normal) ignoraría estas líneas de seguridad.

Esta función gestiona el proceso de autenticación y establecimiento de sesión entre el ESP32 y el servidor Mosquitto. Primero, configura el punto de enlace y el puerto, definiendo además una función de callback para procesar los mensajes entrantes. Al intentar la conexión, el código envía no solo las credenciales de usuario, sino también la configuración del Last Will and Testament (LWT), un mecanismo de seguridad que permite al Broker notificar a otros dispositivos si el ESP32 se desconecta inesperadamente. Una vez establecida la sesión, se amplía el buffer de red a 5 KB para permitir el manejo de paquetes de datos de gran tamaño (ideales para telemetría compleja), se suscribe automáticamente al tópico de control Plc/Esp32 y publica un mensaje de estado en formato JSON para confirmar que el sistema está operativo y correctamente identificado. La función `mqttClient.setCallback(callback)` es fundamental ya que es la encargada de procesar el mensaje de entrada sin esta línea, puedes recibir mensajes, pero tu código no sabría qué función debe procesarlos y los ignoraría.

- Registra la función: Le dice a la librería PubSubClient que la función llamada `callback` (que definiste mas abajo) es la "Elegida".
- Automatización: A partir de ese momento, cada vez que el ESP32 reciba un mensaje MQTT de un tópico al que esté suscrito, la librería interrumpe lo que esté haciendo y ejecuta automáticamente la función `callback` pasándole el mensaje.

---

### Algoritmo 5.13 Función callback()

---

```
void callback(char *topic, byte *payload, unsigned int length) {
 String command = "";
 String str_topic(topic);
 for (int16_t i = 0; i < length; i++) {
 command += (char)payload[i];
 }
 command.trim();
 log("INFO: MQTT Tópico --> " + str_topic);
 log("INFO: MQTT Mensaje --> " + command);

 DynamicJsonDocument JsonCommand(1024);
 DeserializationError error = deserializeJson(JsonCommand, command);
 if (error) {
 mqtt_response("Desconocido", "Desconocido", "",
 "{\"msg\": \"Error, no es un formato JSON!\"}");
 return;
 }
 // Ignorar mensajes propios (evitar bucle infinito)
 if (JsonCommand.containsKey("deviceMqttId")) {
 return;
 }
 if (!JsonCommand.containsKey("method") || !JsonCommand.containsKey("type")) {
 mqtt_response("Desconocido", "Desconocido", "",
 "{\"msg\": \"Error, formato JSON no soportado!\"}");
 return;
 }
 // Manejo de Comandos
 if (strcmp(JsonCommand["method"], "POST") == 0 &&
 strcmp(JsonCommand["type"], "RELAYS") == 0) {
 // {"method": "POST", "type": "RELAYS", "data":{"protocol": "MQTT",
 // "output": "RELAY", "value": false}}
 if (apiPostOnOffRelays(JsonCommand["data"])) {
 if (settingsSave()) {
 mqtt_response(JsonCommand["method"], JsonCommand["type"],
 JsonCommand["data"]["output"], "{\"value\": true}");
 mqtt_publish();
 }
 } else {
 if (settingsSave()) {
 mqtt_response(JsonCommand["method"], JsonCommand["type"],
 JsonCommand["data"]["output"], "{\"value\": false}");
 mqtt_publish();
 }
 }
 }
 else if (strcmp(JsonCommand["method"], "POST") == 0 &&
 strcmp(JsonCommand["type"], "DIMMER") == 0) {
 // {"method": "POST", "type": "DIMMER", "data":{"protocol": "MQTT",
 // "output": "Dimmer", "value": 50}}
 apiPostDimmer(JsonCommand["data"]);
 // respuesta al cliente MQTT
 mqtt_response(JsonCommand["method"], JsonCommand["type"],
 JsonCommand["data"]["output"],
 "{\"value\": " + String(dim) + "}");
 mqtt_publish();
 }
 else if (strcmp(JsonCommand["method"], "POST") == 0 &&
 strcmp(JsonCommand["type"], "RESTART") == 0) {
 // CONFIGURAR EL MQTT MEDIANTE POST
 // {"method": "POST", "type": "RESTART", "origin": "MQTT"}
 // llamar la funcion con los datos
 mqtt_response(JsonCommand["method"], JsonCommand["type"], "",
 F("{\"restart\": true}"));
 delay(100);
 apiPostRestart(JsonCommand["origin"]);
 }
 else if (strcmp(JsonCommand["method"], "POST") == 0 &&
 strcmp(JsonCommand["type"], "RESTORE") == 0) {
 // CONFIGURAR EL MQTT MEDIANTE POST
 // {"method": "POST", "type": "RESTORE", "origin": "MQTT"}
 // llamar la funcion con los datos
 mqtt_response(JsonCommand["method"], JsonCommand["type"], "",
 F("{\"restore\": true}"));
 delay(100);
 apiPostRestore(JsonCommand["origin"]);
 }
 else {
 mqtt_response("Desconocido", "Desconocido", "",
 "{\"msg\": \"Error, no es un comando soportado!\"}");
 }
}
```

---

Esta función actúa como el centro de control y despacho de comandos del sistema, encargándose de procesar de manera asíncrona todos los mensajes entrantes desde el Broker MQTT. El código transforma el flujo de bytes recibido en un objeto estructurado mediante la librería ArduinoJ-

son, implementando primero filtros de seguridad para validar el formato, descartar mensajes mal formados y prevenir bucles infinitos al ignorar ecos del propio dispositivo. Una vez validada la estructura, el callback opera como un intérprete de protocolos API sobre MQTT, discriminando entre diferentes métodos (como POST) y tipos de carga (RELAYS, DIMMER, RESTART o RESTORE); esto permite ejecutar acciones de control físico, actualizar el estado de actuadores, guardar configuraciones en la memoria no volátil y emitir respuestas de confirmación en tiempo real hacia la base de datos o el cliente, garantizando una arquitectura de control bidireccional robusta y escalable.

3 dentro de 5.13 hay dos funciones `mqtt_response` 5.14 y `mqtt_publish` 5.15.

---

**Algoritmo 5.14** Función `mqtt_response()`

---

```
void mqtt_response(String method, String type, String msg, String value) {
 String data = "";
 DynamicJsonDocument jsonDoc(10240);
 DynamicJsonDocument jsonData(10240);
 deserializeJson(jsonData, value);
 jsonDoc["method"] = method;
 jsonDoc["type"] = type;
 JsonObject dataObj = jsonDoc.createNestedObject("data");
 dataObj["msg"] = msg;
 dataObj["value"] = jsonData;
 serializeJson(jsonDoc, data);
 String topic = "Plc/Esp32"; // Simulando PathMqttTopic("response")
 mqttClient.publish(topic.c_str(), data.c_str()); }

```

---

Esta función `mqtt_response` se encarga de construir y enviar una respuesta estructurada en formato JSON a través de MQTT, normalmente como reacción a un comando o solicitud recibida por el ESP32. Primero crea dos documentos JSON dinámicos: uno para el mensaje final (`jsonDoc`) y otro para interpretar el contenido recibido en el parámetro `value`. Luego deserializa `value` para convertirlo en un objeto JSON válido, lo inserta dentro del campo `data.value` y añade metadatos como el método (`method`), el tipo de mensaje (`type`) y un mensaje descriptivo (`msg`). Finalmente, el objeto completo se serializa a una cadena y se publica en el tópico MQTT `Plc/Esp32`, permitiendo que otros sistemas (backend, dashboards o servicios de control) reciban una respuesta clara, estructurada y consistente sobre el estado o resultado de la operación ejecutada por el dispositivo.



---

**Algoritmo 5.15** Función `mqtt_publish()`

---

```
void mqtt_publish() {
 String topic = String(mqtt_topic);
 mqtt_data = Json();
 mqttClient.publish(topic.c_str(), mqtt_data.c_str());
 mqtt_data = "";
}
```

---

Esta función `mqtt_publish` se encarga de publicar periódicamente el estado o los datos actuales del ESP32 en el broker MQTT. Primero construye el tópico de publicación a partir de la variable `mqtt_topic`, asegurando que el mensaje se envíe al canal correcto. Luego genera el contenido del mensaje llamando a la función `Json()`, que encapsula la información del dispositivo (como lecturas de sensores, estados de relés o indicadores del sistema) en formato JSON. Ese mensaje se publica usando el cliente MQTT y, una vez enviado, la variable `mqtt_data` se limpia para liberar memoria y evitar reutilizar datos antiguos. De esta forma, la función mantiene una comunicación ordenada y consistente entre el ESP32 y los sistemas suscriptores, como el backend o el panel de monitoreo.

4 La función `Json` es creada con el propósito de enviar la información asociada al dispositivos 5.16.

---

**Algoritmo 5.16** Json()

---

```
String Json() {
 String response;
 DynamicJsonDocument jsonDoc(3000);
 readSensor();
 jsonDoc["deviceMqttId"] = mqtt_cloud_id;
 jsonDoc["deviceSerial"] = deviceID();
 jsonDoc["deviceManufacturer"] = device_manufacturer;
 jsonDoc["deviceFwVersion"] = device_fw_version;
 jsonDoc["deviceHwVersion"] = device_hw_version;
 jsonDoc["deviceSdk"] = ESP.getSdkVersion();
 JsonObject dataObj = jsonDoc.createNestedObject("data");
 dataObj["deviceRamSizeKB"] = ESP.getHeapSize() / 1024;
 dataObj["deviceRamAvailableKB"] = ESP.getFreeHeap() / 1024;
 dataObj["deviceSpiffsSizeKB"] = SPIFFS.totalBytes() / 1024;
 dataObj["deviceSpiffsUsedKB"] = SPIFFS.usedBytes() / 1024;
 dataObj["deviceActiveTimeSeconds"] = longTimeStr(millis() / 1000);
 dataObj["deviceCpuClockMhz"] = getCpuFrequencyMhz();
 dataObj["deviceFlashSizeMB"] = ESP.getFlashChipSize() / (1024.0 * 1024);
 // Valores Hardware Reales
 dataObj["deviceRelay1Status"] = RELAY1_STATUS ? true : false;
 dataObj["deviceRelay2Status"] = RELAY2_STATUS ? true : false;
 dataObj["deviceDimmer"] = dim; dataObj["deviceCpuTempC"] = TempCPUValue();
 dataObj["deviceADCValue"] = adcValue;
 dataObj["deviceRestarts"] = device_restart;
 dataObj["wifiRssiStatus"] = WiFi.RSSI();
 dataObj["wifiQuality"] = getRSSIasQuality(WiFi.RSSI());
 dataObj["wifiIPv4"] = ipStr(WiFi.localIP());
 jsonDoc["jsonVersion"] = "1.0.0";
 serializeJson(jsonDoc, response); return response; }
```

---

Esta función Json se encarga de recopilar el estado completo del ESP32 y serializarlo en un mensaje JSON listo para ser enviado, normalmente vía MQTT. Al inicio ejecuta readSensor() para actualizar los valores físicos más recientes, como el ADC. Luego construye un objeto JSON que incluye información de identificación del dispositivo (ID MQTT, número de serie, fabricante y versiones de firmware, hardware y SDK), junto con métricas del sistema como uso de memoria RAM, almacenamiento SPIFFS, tiempo activo, frecuencia de CPU y tamaño de la flash. Dentro del bloque data también se agregan los valores reales de hardware, como el estado de los relés, el dimmer, la temperatura del CPU, la lectura ADC y estadísticas de red WiFi (RSSI, calidad de señal e IP). Finalmente, se añade una versión del esquema JSON, se serializa todo a una cadena y se retorna, proporcionando una instantánea completa y estructurada del estado del

dispositivo para monitoreo, diagnóstico o análisis remoto.